

Cuprins

Capitolul 1	Introducere	11
1.1	Contextul proiectului	11
1.2	Descrierea problemei	11
1.3	Tema lucrării	12
Capitolul 2	Obiectivele Proiectului	13
2.1	Obiective generale	13
2.2	Obiective specifice	13
2.2.1	Gestionarea sesiunilor de curs	13
2.2.2	Înregistrarea prezenței prin cod QR	13
2.2.3	Vizualizarea în timp real	14
2.2.4	Gestionarea notelor	14
2.2.5	Istoricul sesiunilor	14
2.2.6	Pagina de administrare	14
2.2.7	Implementarea unui sistem de autentificare	14
2.3	Specificații tehnice	14
2.3.1	Specificații funcționale	14
2.3.2	Specificații non-funcționale	15
Capitolul 3	Studiu Bibliografic	16
3.1	Concepte de bază	16
3.1.1	Sisteme de prezență în mediul universitar	16
3.1.2	Coduri QR	16
3.1.3	Servicii web	17
3.1.4	Interfețe de utilizator	18
3.1.5	Protocolul HTTP și REST API	18
3.1.6	Aplicații de tip SPA	20
3.1.7	Protocoale de comunicare de timp real	20
3.1.8	Procesarea asincronă și cozile de mesaje	21
3.1.9	Containerizarea aplicațiilor	22
3.1.10	Baze de date relaționale	23
3.2	Arhitecturi web	24

3.2.1	Arhitectura monolitică	24
3.2.2	Modelul client-server	24
3.2.3	Microservicii	25
3.3	Posibile soluții și abordarea aleasă	26
Capitolul 4 Analiză și Fundamentare Teoretică		28
4.1	Limbajul de programare Java	28
4.1.1	Apache Maven	28
4.1.2	Librăria Lombok	29
4.2	Cadrul de lucru Spring	29
4.2.1	Spring Boot	30
4.2.2	Spring Web	30
4.2.3	Spring WebSocket	30
4.2.4	Spring Data JPA	31
4.3	Librăria ZXing	31
4.4	Platforma Docker	32
4.5	Baza de date PostgreSQL	32
4.6	RabbitMQ	33
4.7	Serviciul Nginx	33
4.8	Biblioteca React	34
Capitolul 5 Proiectare de Detaliu și Implementare		35
5.1	Detalii de proiectare	35
5.1.1	Cazuri de utilizare	35
5.1.2	Diagrame de secvență	38
5.1.3	Diagrama de sistem	42
5.1.4	Arhitectura bazei de date	43
5.1.5	Structura aplicației	46
5.1.6	Arhitectura serverului SpringBoot	47
5.2	Implementarea serverului SpringBoot	50
5.2.1	Dependențele aplicației	50
5.2.2	Setările aplicației	50
5.2.3	Stratul de controlere	51
5.2.4	Stratul de servicii	54
5.2.5	Stratul de accesare a datelor	54
5.2.6	Autentificarea și autorizarea	55
5.2.7	Configurarea și utilizarea componentei RabbitMQ	56
5.2.8	Configurarea și utilizarea protocolului WebSocket	57
5.3	Implementarea interfeței de utilizator	58
5.3.1	Biblioteci externe utilizate	58
5.3.2	Configurarea serverului	58
5.3.3	Paginile aplicației	59

5.3.4	Rutarea paginilor și protejarea rutelor	60
5.3.5	Gestionarea sesiunii utilizatorului	61
5.3.6	Actualizarea datelor în timp real	62
5.4	Implementarea infrastructurii	63
5.4.1	Containerizarea serverelor	63
5.4.2	Crearea rețelei Docker	65
Capitolul 6	Testare și Validare	66
6.1	Testarea unitară	66
6.2	Testarea de integrare	67
6.3	Testarea de performanță	67
6.4	Testarea interfeței de utilizator	68
Capitolul 7	Manual de Instalare și Utilizare	69
7.1	Manual de instalare	69
7.2	Utilizarea aplicației	70
Capitolul 8	Concluzii	74
8.1	Rezultate obținute	74
8.2	Direcții de dezvoltare	75
Bibliografie		77

Capitolul 1. Introducere

1.1 Contextul proiectului

Digitalizarea proceselor administrative din universități a devenit o preocupare tot mai mare pe măsură ce instituțiile de învățământ încearcă să se modernizeze. Una dintre activitățile care se face simțită direct în calitatea sistemului educațional este gestionarea prezenței la cursuri și laboratoare. În forma ei tradițională, aceasta consumă timp valoros din ora de curs și introduce erori datorate completării manuale ale foilor de prezență.

Sistemele clasice de prezență se bazează fie pe verificarea fiecărui student în parte, fie pe o foaie de prezență pe care fiecare student se semnează. Ambele metode au dezavantaje destul de clare: consumă mult timp, pot duce la confuzii sau chiar falsificări și nu îi oferă profesorului acces imediat la date centralizate. Având în vedere raspândirea telefoanelor inteligente în rândul studenților și evoluția tehnologiilor web în timp real, a devenit posibilă conceperea unor soluții care să automatizeze acest proces. Astfel, prezența poate fi înregistrată mai sigur, iar datele ajung pe loc într-un sistem informatic modern.

Proiectul de față propune o platformă universitară de înregistrare a prezenței bazată pe coduri QR. Ideea este să se înlocuiască demersul tradițional manual cu unul digital, transparent și ușor de verificat. Platforma este gândită ca un sistem complet: de la deschiderea sesiunii de curs de către profesor, până la vizualizarea prezenței în timp real și gestionarea notelor.

1.2 Descrierea problemei

Problema principală pe care o adresează proiectul este lipsa unui sistem eficient și fiabil, care să funcționeze în timp real, pentru înregistrarea și monitorizarea prezenței studenților la orele din învățământul superior.

Mai concret, se observă următoarele probleme:

- Înregistrarea prezenței: cum poate fi făcută rapid, fără să întrerupă cursul, și cum se poate garanta că doar studenții prezenți fizic apar ca prezenți.
- Reducerea fraudei: un student nu trebuie să poată înregistra prezența altui student, iar înregistrarea de mai multe ori pentru aceeași sesiune trebuie oprită la nivel de sistem.

- Lipsa unui sistem centralizat modern: profesorii nu au un instrument unic în care să țină atât prezența, cât și notele. De obicei datele sunt împărțite prin fișiere separate sau introduse manual în sisteme care nu comunică între ele.

Soluția propusă încearcă să rezolve toate aceste probleme printr-o arhitectură în trei niveluri. Aceasta combină o interfață de utilizator cu un server și cu un mecanism de mesagerie asincronă pentru procesarea concurentă a înregistrărilor.

1.3 Tema lucrării

Lucrarea propune proiectarea și implementarea unei platforme universitare de gestiune a prezenței și a notelor, construită cu ajutorul codurilor QR și a comunicației în timp real. Sistemul este gândit pentru mediul academic și acoperă întreaga gestionare a prezențelor și a notelor: de la deschiderea unei sesiuni de curs de către profesor, la înregistrarea prezenței de către studenți prin scanarea unui cod QR, până la vizualizarea datelor într-un singur loc și introducerea notelor.

Alegerea acestei teme pleacă de la câteva observații din mediul universitar:

- Ineficiența metodelor tradiționale: verificarea individuală a studenților durează câteva minute din ora de curs. Înmulțit cu numărul de cursuri și de semestre, timpul pierdut se adună și devine semnificativ, atât pentru profesori cât și pentru studenți.
- Lipsa datelor centralizate în timp real: foile de prezență pe hârtie trebuie introduse ulterior în sisteme informatice, ceea ce înseamnă întârzieri și posibile greșeli de transcriere.
- Potențialul telefoanelor mobile: aproape toți studenții au un telefon care poate citi un cod QR. Această infrastructură există deja și poate fi folosită fără costuri în plus pentru instituție.
- Nevoia unui sistem integrat: platformele și uneltele existente gestionează separat prezența și notele, iar profesorii ajung să folosească mai multe instrumente în același timp. Un sistem unic simplifică munca de zi cu zi.

Capitolul 2. Obiectivele Proiectului

2.1 Obiective generale

Obiectivul principal al proiectului este proiectarea și implementarea unei platforme web complete pentru gestionarea prezenței pe bază de coduri QR și a notelor studenților. Platforma are suport pentru mai multe roluri (profesor, student, administrator) și folosește mesageria asincronă pentru procesarea concurentă a înregistrărilor.

Platforma trebuie să fie:

- Funcțională: să acopere întregul demers al acțiunilor, de la crearea sesiunii până la consultarea situației școlare.
- Fiabilă: să garanteze că fiecare prezență este înregistrată o singură dată și că datele sunt corecte.
- Reactivă: să ofere actualizări în timp real.
- Ușor de utilizat: interfața să fie accesibilă atât de pe desktop, cât și de pe telefon.

2.2 Obiective specifice

2.2.1 Gestionarea sesiunilor de curs

Profesorul trebuie să poată crea o sesiune de curs pentru un curs și o dată anume. La creare, sistemul generează automat un cod QR unic, care conține un cod de sesiune cu durată limitată. Profesorul afișează codul QR, iar studenții îl scanează ca să își înregistreze prezența.

2.2.2 Înregistrarea prezenței prin cod QR

Studentul scanează codul QR cu camera telefonului, este dus către o pagină web de înregistrare și își confirmă identitatea. Sistemul validează codul, verifică unicitatea înregistrării (un student nu se poate înregistra de două ori la aceeași sesiune) și actualizează prezența în baza de date.

Constrângerea de unicitate este impusă la nivelul bazei de date și prin coada de mesaje, care pune înregistrările în ordine și elimină duplicatele atunci când vin foarte multe cereri deodată.

2.2.3 Vizualizarea în timp real

Profesorul vede lista studenților prezenți actualizată în timp real, fără să fie nevoie să reîncarce pagina manual. Actualizările se transmit prin WebSocket, așa că timpul dintre momentul în care un student se înregistrează și apariția numelui său pe ecranul profesorului este foarte scurt.

2.2.4 Gestionarea notelor

Platforma conține și un sistem de note, prin care profesorii pot introduce note pentru studenți la nivelul cursului. Studenții își pot consulta propriile note și prezețe printr-o interfață dedicată.

2.2.5 Istoricul sesiunilor

Atât profesorii, cât și studenții au acces la un istoric al sesiunilor anterioare, cu detalii despre prezență și note. Astfel se poate urmări evoluția participării la cursuri pe parcursul semestrului.

2.2.6 Pagina de administrare

Rolul de administrator permite gestionarea utilizatorilor (studenți, profesori), a cursurilor, a notelor și a prezențelor. Acesta oferă o vedere de ansamblu și control asupra întregii platforme.

2.2.7 Implementarea unui sistem de autentificare

La conectare, utilizatorul își introduce datele de acces, iar sistemul îi verifică identitatea, stabilește rolul (profesor, student sau administrator) și nivelul corespunzător de acces.

2.3 Specificații tehnice

2.3.1 Specificații funcționale

- Sistemul trebuie să suporte cel puțin trei roluri de utilizator: profesor, student și administrator.
- Sistemul trebuie să proceseze cu succes un număr mare de cereri în paralel.
- Codul QR trebuie să fie unic per sesiune și să poată fi invalidat după încheierea ei.
- Codul QR trebuie să expire automat după o perioadă de timp specificată.
- Înregistrarea unui student prezent trebuie să apară pe ecranul profesorului în cel mult o secundă de la confirmare.
- Sistemul trebuie să respingă înregistrările duplicate pentru aceeași sesiune.

2.3.2 Specificații non-funcționale

- Scalabilitate: arhitectura bazată pe RabbitMQ permite scalarea orizontală a componentei care procesează prezența.
- Portabilitate: întreaga soluție este realizată cu Docker și Docker Compose pentru a putea fi rulată pe orice platformă.
- Menținabilitate: separarea clară între interfața de utilizator, serverul care procesează datele și infrastructură (PostgreSQL, RabbitMQ) face mai ușoară evoluția independentă a fiecărei componente.
- Securitate: accesul la resurse este controlat prin coduri de sesiune.
- Robustețe: aplicația validează datele primite de la utilizatori și gestionează erorile, astfel încât intrările incorecte sau incomplete să nu ducă la comportamente nedorite sau la coruperea datelor.
- Utilizabilitate: Platforma este intuitivă și ușor de folosit, atât de profesori cât și de studenți.

Capitolul 3. Studiu Bibliografic

3.1 Concepte de bază

3.1.1 Sisteme de prezență în mediul universitar

Pentru a rezolva problemele sistemelor tradiționale de prezență, au apărut sisteme automate, bazate pe diferite tehnologii de identificare. Se folosesc, printre altele, carduri cu bandă magnetică sau cu cip, etichete cu identificare prin radiofrecvență (în engleză: Radio-Frequency Identification sau RFID), date biometrice precum amprenta sau recunoașterea facială, și mai recent coduri QR scanate cu telefonul. Fiecare abordare are un compromis între cost, viteză și siguranța identificării. Soluțiile biometrice sunt greu de fraudat, însă cer echipamente dedicate și implică probleme legate de confidențialitatea datelor. Cardurile sunt mai ieftine, dar pot fi împrumutate între studenți.

În lucrările care studiază sisteme de prezență pe bază de coduri QR se observă că acestea oferă un echilibru bun: nu necesită cititoare speciale, fiindcă aproape fiecare student are deja un telefon inteligent cu cameră, iar costul de implementare rămâne scăzut [1]. Soluțiile mai noi merg mai departe și leagă codul QR de o sesiune cu durată limitată, tocmai pentru a îngreuna copierea și distribuirea codului în afara sălii [2].

3.1.2 Coduri QR

Codul QR (Quick Response) este un cod de bare bidimensional, format dintr-o matrice de module negre și albe așezate pe un fond pătrat. A fost creat în anul 1994 de compania japoneză Denso Wave, inițial pentru urmărirea pieselor în industria auto [3]. Spre deosebire de codurile de bare clasice, care codifică informația pe o singură dimensiune, codul QR folosește ambele dimensiuni și astfel poate stoca semnificativ mai multe date în același spațiu.

Specificația codului este standardizată la nivel internațional prin documentul ISO/IEC 18004 [4]. Un cod QR poate conține date numerice, alfanumerice, octeți sau caractere Kanji. Pentru o aplicație web interesează în special modul în care un cod QR poate stoca un text scurt, de obicei o adresă URL care duce telefonul direct către o pagină. În acest fel studentul nu trebuie să tasteze nimic: scanează codul, iar telefonul deschide pagina de înregistrare.

O caracteristică importantă este corecția de erori. Standardul prevede patru niveluri, notate "L", "M", "Q" și "H", care permit reconstituirea datelor chiar dacă o parte din cod este deteriorată sau acoperită. Aceasta face codul tolerant la zgârieturi, reflexii sau la o afișare de calitate mai slabă.

Cele trei pătrate mari din colțuri se numesc tipare de detecție a poziției (în engleză: *finder patterns*) și îi permit aparatului cititor să recunoască orientarea codului indiferent de unghiul din care este privit.

3.1.3 Servicii web

Un serviciu web se referă la o componentă software care expune funcționalitate și informații prin rețea, de obicei utilizând protocolul HTTP (Vezi 3.1.5), astfel încât alte aplicații să o poată folosi fără să cunoască detaliile interne de implementare. Ideea centrală este compatibilitatea: aplicațiile care utilizează acest serviciu web pot fi scrise în limbaje diferite și pot rula pe mașini diferite, singura regulă fiind respectarea aceluiași protocol de comunicare.

De-a lungul timpului s-au impus mai multe stiluri de servicii web. Modelul mai vechi, bazat pe SOAP (în engleză: *Simple Object Access Protocol*), folosește mesaje XML (*Extensible Markup Language*) cu o structură strictă și un fișier de descriere a serviciului. Acest model este puternic tipizat, dar greu de utilizat, având nevoie de multă configurare. Se mai folosește și în prezent, dar tot mai rar. Modelul mai nou și mai răspândit a ajuns să fie cel de tip REST (Vezi 3.1.5) care se bazează direct pe mecanismele protocolului de comunicare HTTP.

Construirea unui serviciu web pornind doar de la protocolul HTTP presupune mult cod generic, cum ar fi gestionarea unei conexiuni la o bază de date. Din acest motiv au apărut cadre de lucru (în engleză: *frameworks*) care oferă aceste mecanisme generice deja construite. Folosind aceste cadre de lucru, dezvoltarea aplicațiilor se rezumă la implementarea particularităților acestora. Câteva dintre cele mai folosite astfel de cadre de lucru sunt:

- Spring Boot: este un cadru de lucru pentru limbajul Java, construit în jurul ecosistemul Spring [5]. Scopul lui principal este să simplifice pornirea unei aplicații prin configurare automată. Conține un server web inclus și multe module care se adaugă după nevoie. Utilizând acest cadru de lucru se poate construi un serviciu web cu foarte puțină configurare inițială (Vezi 4.2.1).
- Django: este un cadru de lucru complet pentru limbajul Python [6]. Acesta vine cu multe componente integrate folosite pentru dezvoltarea unui serviciu web. Configurarea acestui cadru de lucru este foarte ușoară. Avantajul este viteza de dezvoltare iar dezavantajul este folosirea unei structuri mai rigide.
- ASP.NET Core: este un cadru de lucru dezvoltat de compania Microsoft pentru platforma .NET, folosind de obicei limbajul C#. Oferă performanță bună și o compatibilitate mare cu celelalte unelte din ecosistemul Microsoft.
- FastAPI: este un cadru de lucru modern și minimalist pentru limbajul Python [7]. Se poate utiliza la servicii mici sau la dezvoltarea rapidă de prototipuri.

3.1.4 Interfețe de utilizator

Interfețele de utilizator sunt un ansamblu de componente care servesc ca și punct de contact dintre oameni și calculatoare. Prin ele utilizatorii văd informațiile pe care sistemul le pune la dispoziție și pot să își exprime acțiunile prin elementele periferice ale calculatorului: click-uri, completări ale câmpurilor text sau navigat între pagini. Un scop important al interfețelor de utilizator este să ascundă complexitatea din spatele aplicațiilor și să lase utilizatorul să folosească aplicațiile fără să fie nevoit să înțeleagă cum funcționează acestea în detaliu.

În aplicațiile web, interfața se construiește de obicei pe trei tehnologii care lucrează împreună. Prima este HTML (Hypertext Markup Language), un limbaj de marcare prin care se descrie structura paginii: titluri, paragrafe, liste, formulare și legături [8]. HTML descrie ce elemente există pe pagină și cum sunt ele organizate. A doua tehnologie este CSS (Cascading Style Sheets) care se ocupă de aspectul elementelor: culori, dimensiuni, spațieri, poziționarea elementelor și adaptarea la diferite mărimi de ecran [9]. A treia tehnologie este JavaScript, limbaj de programare prin care pagina devine interactivă. Cu JavaScript se pot procesa acțiunile utilizatorului și se pot modifica elementele paginii în funcție de acestea.

Pe măsură ce aplicațiile web au crescut în complexitate, folosirea limbajului JavaScript a devenit tot mai grea. Din acest motiv au apărut biblioteci și cadre de lucru care oferă o structură bazată pe aceste tehnologii de bază și ușurează dezvoltarea interfețelor moderne de utilizator. Câteva dintre acestea sunt:

- **React:** este o bibliotecă creată de compania Meta pentru construirea interfețelor pe bază de componente [10]. Ideea centrală este împărțirea interfeței în bucăți mici și reutilizabile numite componente, fiecare cu propria logică și propriul aspect. React nu impune un mod strict de organizare a întregii aplicații, ci lasă dezvoltatorului libertatea de a alege biblioteci suplimentare. (Vezi 4.8)
- **Angular:** este un cadru de lucru complet dezvoltat de compania Google, care conține soluții integrate pentru aproape tot ce are nevoie o aplicație web, de la navigare la comunicarea cu serverul și validarea formularelor [11]. Acest cadru de lucru impune o structură clară și folosește limbajul TypeScript, care adaugă funcționalități suplimentare limbajului JavaScript. Este mai greu de utilizat dar are avantajul unei structuri clare.
- **Flutter:** este un cadru de lucru cu un scop mai larg. Folosește limbajul Dart și permite construirea unei singure baze de cod din care se generează aplicații pentru mai multe platforme deodată: mobil, desktop și web. O particularitate a cadrului de lucru Flutter este că își generează singur elementele de interfață, fără a necesita dezvoltarea directă folosind HTML și CSS.

3.1.5 Protocolul HTTP și REST API

Protocolul HTTP (Hypertext Transfer Protocol) este sistemul pe care se bazează schimbul de date între aplicații software [12]. Funcționează după modelul cerere-răspuns: o aplicație client

trimite o cerere către un server (serviciu web), iar serverul întoarce un răspuns. Fiecare cerere indică o metodă care arată ce fel de operație se dorește de la server și o adresă către resursa serverului. Răspunsul conține un cod care reprezintă starea cererii și de cele mai multe ori un corp cu datele cerute sau cu mai multe informații despre cerere sau operațiunea executată de server. Semantica metodelor și a codurilor de stare este descrisă în specificațiile HTTP [13].

Termenul API (în engleză: Application Programming Interface) se referă la un set de reguli prin care două programe comunică între ele. Un API stabilește ce cereri se pot face și ce răspunsuri se pot da, fără ca cel care îl folosește să fie nevoit să cunoască felul în care serviciul este construit [14]. În cazul aplicațiilor web, un astfel de API este expus de server și accesat prin HTTP de către alte programe. Comunicarea între un client și un API este ilustrată în figura 3.1

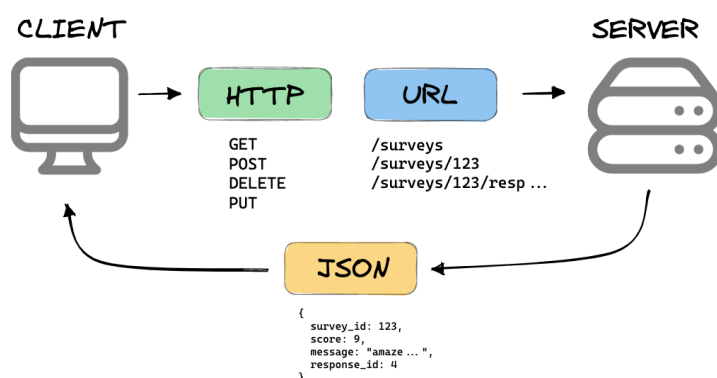


Figura 3.1: Comunicarea client-API [15]

REST (în engleză: Representational State Transfer) este un stil arhitectural introdus de Roy Fielding în anul 2000 [16]. Acesta reprezintă un set de principii care, atunci când sunt respectate, conduc la sisteme distribuite scalabile și ușor de întreținut. Un API care urmează aceste principii se numește în mod obișnuit REST API sau API RESTful.

Principiile de bază sunt următoarele:

- Resurse identificate prin URI [17]: fiecare entitate are o adresă proprie prin care poate fi accesată.
- Operații prin metode HTTP: se folosește semantica metodelor standard, unde "GET" citește o resursă, "POST" creează una nouă, "PUT" o actualizează, iar "DELETE" o șterge [13].
- Comunicare fără stare (în engleză: stateless): fiecare cerere conține toate informațiile necesare procesării, iar serverul nu păstrează contextul între două cereri succesive.
- Reprezentări: aceeași resursă poate fi transmisă în formate diferite, însă în practica modernă se regăsește aproape mereu JSON [18], fiind ușor de citit și de prelucrat atât de către om, cât și de către sistemele de calcul.

Caracterul "stateless" aduce un avantaj direct pentru scalabilitate fiindcă serverul nu memorează nimic despre client între cereri, mai multe instanțe ale aplicației pot trata cereri în paralel, iar o cerere poate ajunge la oricare dintre ele.

3.1.6 Aplicații de tip SPA

O aplicație de tip SPA (Single-Page Application) este o aplicație web care încarcă o singură pagină HTML și apoi actualizează conținutul dinamic, prin JavaScript, fără a mai cere de fiecare dată o pagină nouă de la server [19]. Navigarea între paginile web se simte instantanee, fiindcă numai datele necesare sunt aduse din spate, nu întreaga structură a paginii.

Modelul clasic de aplicație web, denumit aplicație cu mai multe pagini (în engleză: Multi-Page Application sau MPA), funcționează altfel. La fiecare acțiune a utilizatorului, clientul cere serverului o pagină HTML complet nouă, pe care apoi o reîncarcă. Aceasta înseamnă o pauză vizibilă la navigarea între pagini și un trafic mai mare, fiindcă se retrimite și părți care de fapt nu s-au schimbat.

În cazul unei aplicații de tip SPA, separarea dintre interfață și server este mai clară. Serverul este un furnizor de date prin REST API, iar logica de afisare este în întregime pe partea clientului. Această separare are și un dezavantaj: prima încărcare poate fi mai lentă, fiindcă clientul trebuie să descarce tot codul JavaScript al aplicației înainte să afișeze ceva util. După acest moment inițial însă, interacțiunile devin foarte rapide.

3.1.7 Protocoale de comunicare de timp real

Protocolul HTTP pe care se bazează serviciile web obișnuite urmează modelul cerere-răspuns: clientul face o cerere, serverul răspunde, iar conexiunea se închide. Acest model nu este potrivit atunci când serverul trebuie să trimită date către client din proprie inițiativă, fără ca acesta să fi cerut ceva. Clientul poate să ia legătura cu serverul, dar serverul nu poate inițializa conexiunea cu clientul.

O primă soluție istorică a fost interogarea repetată în care clientul întreabă serverul la intervale regulate dacă au apărut date noi. Problema apare la stabilirea intervalului de interogare: dacă intervalul este prea scurt, traficul este foarte mare și serverul este epuizat de resurse, dacă intervalul este prea lung, se economisesc resurse dar informațiile ajung cu întârziere. Există pierderi de performanță în alegerea oricărui interval. O variantă îmbunătățită este interogarea lungă care ține cererea deschisă până apar date, însă rămâne o soluție de compromis.

Soluția modernă este protocolul WebSocket, standardizat prin documentul RFC 6455 [20]. Acesta stabilește o conexiune permanentă, bilaterală, peste o singură conexiune TCP, pornită printr-un mecanism de trecere (în engleză: handshake). Acest mecanism de trecere este inițiat prin HTTP. Odată deschisă conexiunea, ambele părți pot trimite mesaje oricând, fără cererile și antetele repetate ale protocolului HTTP. Aceasta reduce latența și traficul, ceea ce o face potrivită pentru aplicații de timp real.

Împreună cu WebSocket se folosește adesea un protocol de mesagerie de nivel mai înalt care impune structura mesajelor. STOMP (Simple Text Oriented Messaging Protocol) este un

protocol simplu, orientat pe text, care introduce conceptul de subiecte (în engleză: topics) la care clienții se pot abona [21]. Atunci când serverul publică un mesaj pentru un anumit subiect, toți clienții abonați acelui subiect vor primi mesajul într-un timp foarte scurt. (Vezi 4.2.3)

3.1.8 Procesarea asincronă și cozile de mesaje

Procesarea asincronă reprezintă modul de lucru în care o cerere nu este tratată imediat, în întregime, ci este preluată și pusă deoparte pentru a fi procesată mai târziu. Astfel, componenta care primește cererea poate să răspundă foarte repede, fără să aștepte ca toată procesarea să se finalizeze.

Mecanismul prin care se realizează des acest lucru este coada de mesaje (în engleză: message queue). O coadă de mesaje este o structură intermediară în care un producător depune mesaje, iar un consumator le preia și le procesează, fără ca cei doi să fie nevoiți să comunice direct sau în același timp. De exemplu, dacă mai mulți clienți fac cereri unui singur server, serverul preia fiecare cerere la momentul când aceasta a fost făcută și o pune într-o coadă, apoi rezolvă fiecare cerere pe rând, în ordine. Clienții și serverul nu trebuie să fie prezenți în același moment. Acest tipar de integrare prin mesaje este folosit tocmai pentru acest tip de conexiune pe care o introduce între componente [22]. Diagrama unui sistem de comunicare între un client, un API și o coadă de mesaje ca intermediar este prezentată în figura 3.2

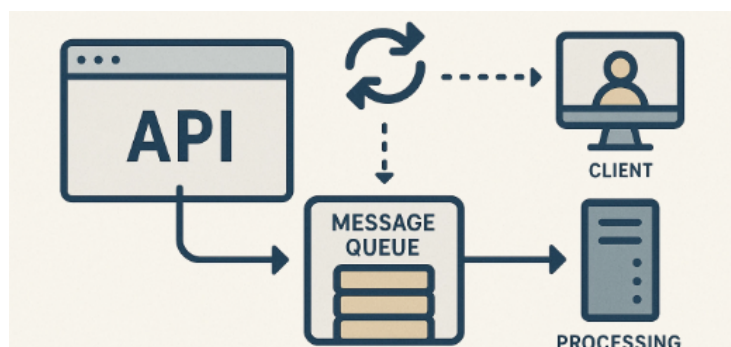


Figura 3.2: Comunicarea folosind coada de mesaje [23]

Avantajele acestui model sunt multe. În primul rând, decuplarea: producătorul și consumatorul nu depind unul de celălalt, dacă se execută multe cereri deodată, ele se adună în coadă și sunt procesate pe rând, fără a suprasolicita sistemul. Fiabilitatea este un alt mare avantaj: o coadă poate păstra mesajele chiar dacă serverul repornește, astfel încât nimic să nu se piardă. Pe lângă acestea, scalabilitatea este un alt atribut important al acestor sisteme. În practica modernă, programele de tip consumator sunt scalate automat, numărul instanțelor de tip consumator este direct proporțional cu numărul de mesaje din coadă pentru a spori viteza de procesare.

Există mai multe sisteme care implementează ideea de coadă de mesaje. Deși toate mută mesaje de la un producător la un consumator, fiecare are particularități și scopuri diferite:

- RabbitMQ: este un broker de mesaje clasic, care implementează protocolul AMQP (Advanced Message Queuing Protocol) [24]. Funcționează după modelul în care brokerul este

inteligent, adică el se ocupă de dirijarea mesajelor către cozile potrivite. Un mesaj este de obicei șters din coadă imediat ce a fost preluat și confirmat de consumator. Acest model se potrivește bine pentru distribuirea sarcinilor de lucru între mai mulți consumatori și pentru comunicarea între componentele unei aplicații (Vezi 4.6).

- Apache Kafka: este un broker în care mesajele nu sunt șterse după ce sunt citite, ci sunt scrise într-un jurnal ordonat și păstrate o perioadă de timp, indiferent dacă au fost sau nu consumate. Fiecare consumator ține singur evidența poziției la care a ajuns, ceea ce îi permite să recitească mesajele mai vechi dacă este nevoie. Kafka este construit pentru volume foarte mari de date și se folosește mai ales în prelucrarea datelor despre evenimente și în construirea conductelor de date (în engleză: data pipelines), fiind mai greu de configurat și de operat decât un broker clasic.
- Azure Service Bus: este un serviciu administrat și oferit în cloud de Microsoft. În acest caz nu mai există un server propriu de întreținut, infrastructura fiind gestionată de furnizor iar plata este făcută în funcție de utilizare. Ca model de operare seamănă cu un broker clasic, oferind cozi și un mecanism de publicare-abonare. Se folosește de obicei în aplicațiile care rulează în ecosistemul Azure pentru a evita administrarea propriei infrastructuri de mesagerie.

3.1.9 Containerizarea aplicațiilor

Containerizarea se referă la procedura de a împacheta o aplicație împreună cu tot ce are nevoie ca să ruleze, adică bibliotecile, fișierele de configurare și celelalte dependențe, într-o singură unitate numită container. Ideea de la care pornește acest concept este o problemă des întâlnită în dezvoltarea software: o aplicație poate să funcționeze pe un calculator dar poate să nu funcționeze pe un server sau pe alt calculator din cauza configurărilor locale, pot să lipsească anumite unelte sau versiunile anumitor biblioteci să difere. Containerul rezolvă acest lucru pentru că include mediul de execuție complet. În acest fel aplicația se comportă la fel indiferent de mașina pe care rulează și este un proces independent.

Pentru a înțelege beneficiile pe care le are containerizarea, este utilă comparația cu mașinile virtuale. O mașină virtuală simulează un calculator întreg, cu propriul sistem de operare. Aceasta înseamnă un consum mare de resurse și un timp de pornire mare. Containerele, în schimb, folosesc în comun nucleul (în engleză: kernel) sistemului de operare gazdă și izolează doar procesele și spațiul de fișiere ale aplicației. Astfel un container consumă mult mai puține resurse și este mult mai ușor de rulat decât o mașină virtuală.

Docker este unealta clasică care a făcut populară ideea de containerizare și a devenit standardul în acest domeniu [25]. Un concept central al Docker este fișierul de descriere numit "Dockerfile" în care se scriu pașii necesari pentru construirea unui container: de la ce imagine de bază se pornește, ce dependențe se instalează și ce comandă se execută la pornire. Din acest fișier se construiește o imagine Docker care este un șablon al aplicației. Dintr-o imagine se pot porni apoi unul sau mai multe containere identice. Imaginile pot fi păstrate și distribuite prin registre

publice sau private, ceea ce ușurează mult mutarea aplicației de pe un calculator pe un server sau pe alt calculator. (Vezi 4.4)

O aplicație modernă este rareori formată dintr-un singur container. De cele mai multe ori sunt necesare mai multe componente care rulează în paralel, de exemplu serverul aplicației, baza de date și un broker de mesaje sunt separate în containere diferite. Pentru a porni și a lega aceste containere între ele se folosește unealta "Docker Compose", care descrie întreaga configurație într-un singur fișier [26]. În acest fișier se definesc serviciile, rețeaua dintre ele și volumele de date, iar pornirea tuturor componentelor se face printr-o singură comandă.

Atunci când numărul de containere crește foarte mult și acestea trebuie distribuite pe mai multe servere, gestionarea lor manuală devine grea. Pentru aceste situații se folosesc unelte de orchestrare, dintre care cea mai cunoscută este Kubernetes [27]. Acestea se ocupă automat de pornirea containerelor, de repornirea lor în caz de eroare și de scalarea numărului de instanțe în funcție de cerere sau de anumite evenimente.

3.1.10 Baze de date relaționale

O bază de date relațională se referă la un mod de organizare a datelor în tabele formate din rânduri și coloane, în care legăturile dintre date sunt exprimate prin valori comune, nu prin pointeri sau structuri fizice. Modelul relațional a fost propus de Edgar F. Codd în anul 1970 și a rămas de atunci baza majorității sistemelor de gestiune a datelor [28].

Fiecare tabel reprezintă un tip de entitate, iar fiecare rând o instanță a acelei entități. Coloanele descriu atributele, iar o cheie primară identifică unic fiecare rând. Legăturile dintre tabele se fac prin chei străine, care leagă un rând dintr-un tabel de un rând din alt tabel. Pe lângă acestea, se pot defini constrângeri care impun reguli asupra datelor, cum ar fi constrângerea de unicitate care interzice valori duplicate într-o coloană sau într-o combinație de coloane.

Limbajul standard prin care se interoghează aceste baze de date este SQL (Structured Query Language). Un concept important este cel de tranzacție: un grup de operații care se execută ca un tot unitar. Tranzacțiile respectă în mod tradițional proprietățile ACID (atomicitate, consistență, izolare, durabilitate) care garantează că datele rămân corecte chiar dacă mai multe operații se desfășoară în același timp.

Există mai multe sisteme de gestiune a bazelor de date relaționale. Deși toate respectă modelul relațional și limbajul SQL, fiecare are particularități și se potrivește mai bine pentru anumite situații:

- PostgreSQL: este un sistem gratuit și cu sursă deschisă (în engleză: open-source), cunoscut pentru respectarea riguroasă a standardelor SQL și pentru fiabilitatea sa [29]. Pe lângă tipurile clasice de date, PostgreSQL poate să gestioneze tipuri de date mai complexe, cum ar fi documentele JSON sau datele geometrice, și poate fi extins cu module suplimentare. Se folosește în aplicațiile web moderne, mai ales acolo unde corectitudinea datelor și tranzacțiile complexe contează (Vezi 4.5).
- MySQL: este tot un sistem cu sursă deschisă, deținut în prezent de Oracle, și unul dintre cele mai răspândite din lume [30]. A devenit popular fiindcă este ușor de instalat și rapid la

operațiile de citire, motiv pentru care apare des în spatele site-urilor și al aplicațiilor web cu trafic mare.

- Oracle Database: este un sistem comercial al companiei Oracle, folosit mai ales în mediul de afaceri, în bănci și în sisteme mari de gestiune. Oferă funcții avansate și performanță bună la volume foarte mari de date, dar vine cu un cost de licențiere ridicat și cu o administrare mai complicată.
- SQLite: este un sistem cu sursă deschisă care gestionează toată baza de date într-un singur fișier, fără un server separat care să ruleze în fundal. Din acest motiv SQLite este foarte ușor de folosit și foarte ușor de configurat. Se folosește mai ales în aplicațiile mobile sau foarte mici, unde datele nu sunt foarte multe și traficul este redus

3.2 Arhitecturi web

Arhitectura reprezintă felul în care sunt împărțite componentele unei aplicații și regulile prin care acestea comunică. Aceasta influențează în mod direct cât de ușor poate fi întreținută aplicația, cum se comportă aplicația la un trafic mare de date și cât de simplu pot evolua părțile ei în mod independent.

3.2.1 Arhitectura monolitică

Arhitectura monolitică reprezintă modelul în care întreaga aplicație este construită și livrată ca o singură unitate. Tot codul, de la interfață la logica de procesare și la accesul la date, trăiește într-un singur proiect și rulează ca un singur proces.

Acest model are avantaje clare, mai ales pentru proiectele mici. Este simplu de înțeles, ușor de dezvoltat la scară mică și simplu de pus în funcțiune, fiindcă este vorba de un singur proces. Pentru aplicații mici sau medii, monolitul este de multe ori cea mai bună alegere de arhitectură.

Problemele la această arhitectură apar pe măsură ce aplicația se dezvoltă tot mai mult. Un monolit mare devine greu de înțeles și de modificat, fiindcă părțile lui ajung strâns legate între ele. O schimbare mică într-o zonă a codului poate afecta zone neașteptate și poate avea efecte secundare nedorite. Pe lângă acestea, scalarea se face în bloc: dacă doar o componentă este suprasolicitată, trebuie multiplicată întreaga aplicație sau trebuie schimbată logica fundamentală a aplicației.

3.2.2 Modelul client-server

Arhitectura bazată pe modelul client-server reprezintă un mod de organizare a aplicațiilor în care sarcinile sunt împărțite între două roluri principale. Pe de o parte se află serverul, care deține resursele și pune la dispoziție servicii, iar pe de altă parte clientul, care cere aceste resurse și servicii și le folosește. Comunicarea dintre cele două se face prin rețea, de obicei după modelul cerere-răspuns, urmând protocolul HTTP (vezi 3.1.5).

Un server poate să răspundă la cererile mai multor clienți în același timp, fără ca aceștia să știe unii de alții. De asemenea, un client nu trebuie să cunoască felul în care serverul este construit în detaliu, ci doar regulile prin care se face schimbul de mesaje. Această separare a rolurilor aduce un avantaj clar: fiecare parte poate fi dezvoltată și modificată separat, atâta timp cât regulile de comunicare rămân aceleasi.

Tot din această separare reiese și modul de scalare. Serverul, fiind partea care concentrează cererile și procesează datele, poate fi multiplicat în mai multe instanțe atunci când traficul crește, fără a fi nevoie de vreo modificare la client. Astfel, un singur server poate deservi foarte mulți clienți, în funcție de resursele alocate.

Modelul client-server stă la baza majorității aplicațiilor web de astăzi. Browserul sau aplicația de pe telefon este clientul, iar în spate se află unul sau mai multe servere care păstrează datele și execută logica. Pe acest model se construiesc și arhitecturile pe mai multe nivele, în care clientul, logica de procesare și baza de date sunt despărțite în straturi separate. O astfel de arhitectură este ilustrată în figura 3.3.

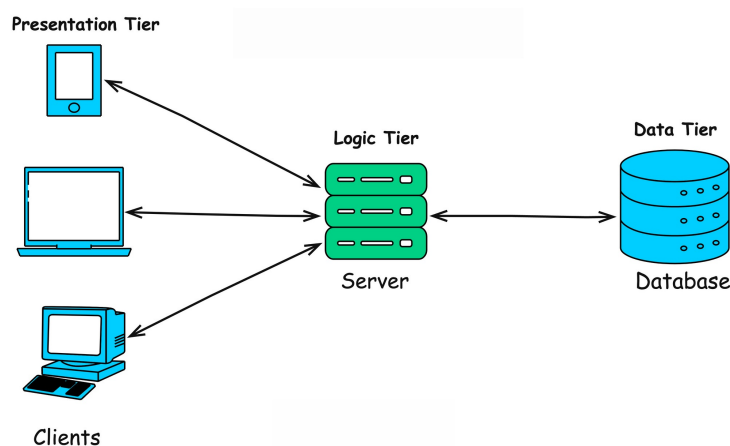


Figura 3.3: Arhitectură de tip client-server pe trei nivele [31]

3.2.3 Microservicii

Arhitectura pe microservicii reprezintă modelul în care o aplicație este împărțită în mai multe aplicații mici și independente, fiecare cu o responsabilitate bine delimitată. Se poate spune că este înversul arhitecturii monolitice: în loc ca tot codul să trăiască într-un singur proces, funcționalitatea este spartă în mai multe procese care rulează separat și comunică între ele prin rețea, de obicei prin HTTP sau prin cozi de mesaje (Vezi 3.1.8) [32].

Fiecare aplicație mică este un serviciu din aplicația totală. Fiecare serviciu se ocupă de o singură zonă a aplicației și își gestionează propriile date și resurse. Astfel, un serviciu poate fi dezvoltat, pus în funcțiune și actualizat fără a fi nevoie să se producă modificări în restul sistemului. De asemenea, fiecare serviciu poate fi scris într-un limbaj diferit și poate folosi o bază de date proprie, atâta timp cât regulile de comunicare între servicii sunt respectate și bine definite.

Avantajul principal al acestui model apare la scalare. Dacă doar o anumită parte a aplicației este suprasolicitată, se multiplică numai serviciul respectiv. Pe lângă aceasta, lucrul în echipă este mai ușor într-o astfel de arhitectură iar defecțiunile dintr-un serviciu nu opresc neapărat tot sistemul, beneficii datorate separării de responsabilități în procese distincte.

Există și dezavantaje în această arhitectură. Un sistem distribuit pe microservicii este mai greu de dezvoltat și de operat. Apar probleme legate de comunicarea prin rețea, de păstrarea consistenței datelor răspândite în mai multe baze de date și de urmărirea unei cereri care trece prin mai multe servicii. Din acest motiv, microserviciile se folosesc de obicei la aplicații mari și complexe, unde câștigul de scalabilitate și de organizare depășește costul suplimentar de infrastructură. Diagrama unui sistem bazat pe microservicii este ilustrată în figura 3.4.

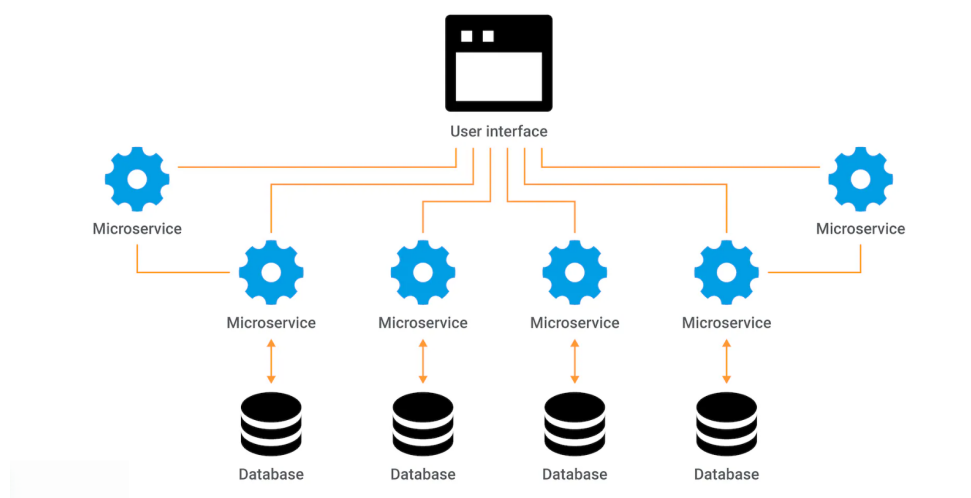


Figura 3.4: Arhitectura bazată pe microservicii [33]

3.3 Posibile soluții și abordarea aleasă

Pentru a aborda obiectivele lucrării există o multitudine de soluții, tehnologii și moduri de implementare. În alegerea soluției s-a ținut cont de următoarele aspecte:

- Aplicația trebuie să fie ușor de adoptat în rândul studenților și a universităților.
- Soluția se construiește pe infrastructura digitală deja existentă, fără costuri sau aparaturi și unelte suplimentare.
- Accentul este pus pe rapiditate și centralizarea datelor, nu pe împiedicarea totală a fraudei.

Pornind de la conceptele și arhitecturile prezentate, se pot alege mai multe combinații pentru problema înregistrării prezenței și a dezvoltării unei aplicații universitare centralizate. O primă variantă ar fi o aplicație monolitică clasică, cu pagini generate pe server și cu prezența

scrisă direct în baza de date la fiecare cerere. Această variantă este simplă, dar nu tratează bine două lucruri: actualizarea în timp real a datelor și valul de cereri simultane de la începutul orei.

O a doua variantă ar fi un sistem distribuit pe microservicii, cu servicii separate pentru prezență, note și utilizatori. Acesta ar fi scalabil și bine organizat, însă complexitatea de dezvoltare și de operare depășesc nevoile reale ale unui proiect de această mărime și de acest scop. Complexitatea infrastructurii nu justifică funcționalitatea aplicației relativ simplă.

Soluția aleasă este la mijlocul acestor ideologii opuse descrise mai sus. Se folosește o arhitectură pe trei niveluri de tip client-server, cu o interfață de utilizator de tip SPA React, un server REST Spring Boot și o bază de date PostgreSQL. Acestei structuri se adaugă două elemente care rezolvă problema actualizării datelor în timp real și problema procesării unui volum mare de cereri într-un timp foarte scurt. Pentru actualizarea în timp real se folosește WebSocket împreună cu protocolul de mesagerie STOMP iar pentru tratarea cererilor simultane se folosește coada de mesaje RabbitMQ, care pune înregistrările în ordine și le procesează una câte una. Detalii relevante despre aceste tehnologii sunt aprofundate în capitolul 4.

Soluția aleasă păstrează simplitatea unei aplicații cu structură clară și bine definită la care se adaugă mecanismele necesare pentru rezolvarea problemelor specifice temei aplicației: comunicarea în timp real și procesarea asincronă.

Capitolul 4. Analiză și Fundamentare Teoretică

4.1 Limbajul de programare Java



Java este un limbaj de programare de nivel înalt, orientat pe obiecte, apărut în anul 1995. Ideea de la care a pornit limbajul este independența față de platformă exprimată prin sloganul ”scrie o dată, rulează oriunde” [34]. Acest lucru se obține prin felul în care codul este executat: codul sursă nu se compilează direct în instrucțiuni pentru un anumit procesor, ci într-un format intermediar numit cod de octeți. Acest cod intermediar este apoi rulat de o mașină virtuală Java care îl traduce în instrucțiuni pentru mașina pe care se află [35]. Astfel același program compilat poate rula pe Windows, Linux sau macOS, fără a fi nevoie de o recompilare. Pe lângă această portabilitate, mașina virtuală se ocupă singură de gestionarea memoriei, eliberând automat memoria ocupată de obiectele care nu mai sunt folosite.

Java este un limbaj puternic tipizat, ceea ce înseamnă că tipul fiecărei variabile este verificat încă de la compilare. Aceasta detectează multe greșeli devreme, înainte ca programul să ruleze. Pe lângă limbajul în sine, un motiv important pentru care Java este des folosit în aplicațiile de tip server este ecosistemul foarte bogat de biblioteci și cadre de lucru construite în jurul lui. Pentru aplicația de față s-a folosit versiunea Java 17, o versiune cu suport pe termen lung.

4.1.1 Apache Maven



Apache Maven este o unealtă care se ocupă de gestionarea dependențelor și de automatizarea procesului de construire a aplicației [36]. Un proiect Java de dimensiuni medii depinde de multe

biblioteci externe, fiecare la rândul ei cu propriile dependențe. Gestionarea manuală a acestora, prin descărcarea fișierelor și adăugarea lor la proiect devine dificilă și predispusă la erori.

Maven funcționează pe baza unui fișier de configurare numit "pom.xml" (Project Object Model) scris în format XML. În acest fișier se declară dependențele de care are nevoie proiectul, fiecare identificată unic printr-un grup, un nume și o versiune. La construirea proiectului, Maven descarcă automat aceste dependențe dintr-un depozit central și le pune la dispoziția proiectului, împreună cu dependențele lor indirecte. De asemenea, Maven structurează procesul de construire în etape bine definite, cum ar fi compilarea, testarea și împachetarea. Aceasta unealta face ca trecerea de la codul sursă la o aplicație gata de rulat să se facă printr-o singură comandă.

4.1.2 Librăria Lombok

Lombok este o bibliotecă pentru limbajul Java al cărei scop este reducerea codului repetitiv [37]. În programarea orientată pe obiect se scriu des aceleași structuri de cod: metode de acces la câmpuri, constructori, metode de comparare sau de afișare. Acest cod nu aduce nimic nou ca logică, dar ocupă spațiu și face citirea claselor mai dificilă.

Lombok intervine în momentul compilării și generează automat acest cod repetitiv, pornind de la niște adnotări puse deasupra claselor sau a câmpurilor. De exemplu, o singură adnotare poate înlocui zeci de linii cu metode de acces. Rezultatul este un cod mai scurt și mai ușor de urmărit în care rămâne vizibilă doar partea care contează.

4.2 Cadrul de lucru Spring



Spring este cel mai răspândit cadru de lucru pentru construirea aplicațiilor în limbajul Java [38]. A apărut în anul 2003 ca o alternativă mai simplă la tehnologiile complicate ale platformei Java EE, iar de atunci a crescut într-un întreg ecosistem de module [39].

Conceptul central în jurul căruia este construit Spring este inversarea controlului realizată prin injectarea dependențelor. În mod obișnuit, fiecare componentă a unei aplicații își creează singură obiectele de care are nevoie. În Spring, lucrurile sunt invers: un container central se ocupă de crearea obiectelor și de furnizarea lor acolo unde sunt cerute. Astfel, componentele nu mai depind de felul în care sunt construite dependențele lor, ci doar de ceea ce acestea oferă. Acest lucru face codul mai ușor de modificat și de întreținut. Ecosistemul Spring este împărțit în module specializate. Modulele folosite în această lucrare sunt prezentate în continuare.

4.2.1 Spring Boot



Spring Boot este modulul cu care s-a construit serverul web al aplicației. Un aspect important îl reprezintă dependențele de tip ”starter”. Un astfel de starter este o dependență care adună la un loc toate bibliotecile necesare pentru o anumită funcționalitate, astfel încât în fișierul de configurare se declară un singur lucru în loc de mai multe.

Potrivit [5], pornind de la bibliotecile prezente în proiect, Spring Boot aplică principiul de convenție peste configurare: se uită la ce se află în proiect și pregătește singur componentele potrivite cu valori implicite rezonabile. De exemplu, prezența unei biblioteci de acces la baza de date duce la pregătirea automată a conexiunii. Valorile care diferă de la un mediu la altul, cum ar fi adresa bazei de date, se țin separat într-un fișier de configurare extern. Acest lucru permite schimbarea configurărilor fără a modifica codul. La final, aplicația este împachetată într-un singur fișier executabil care conține și serverul web, deci poate fi pornit direct.

4.2.2 Spring Web

Spring Web este modulul din ecosistemul Spring care se ocupă de construirea serviciilor web bazate pe protocolul HTTP. Acesta pune la dispoziție tot ce este necesar pentru a expune un REST API: primirea cererilor, dirijarea lor către codul potrivit și trimiterea răspunsurilor înapoi către client.

Conform [38], mecanismul de bază este cel al controlerelor. Un controler este o clasă în care fiecare metodă este legată de o anumită adresă și de o anumită metodă HTTP. Atunci când o cerere ajunge la server, Spring Web se uită la adresă și la metoda HTTP și alege metoda din controler care se potrivește, apelând codul. Legarea aceasta se face declarativ, prin adnotări puse deasupra metodelor, fără să fie nevoie de cod pentru analiza cererilor. De asemenea, modulul se ocupă singur de transformarea datelor: obiectele Java întoarse de metode sunt convertite automat în format JSON pentru răspuns, iar corpul cererilor primite în JSON este transformat înapoi în obiecte Java. Această conversie automată evită muncă repetitivă și predispusă la greșeli.

4.2.3 Spring WebSocket

Spring WebSocket este modulul dedicat implementării comunicării de timp real și a protocoalelor corespunzătoare.

Conform [40], modulul gestionează în spate un broker simplu de mesaje în memoria serverului care ține evidența abonărilor la subiecte și distribuie mesajele către clienții potriviți. Mesaje primite de la clienți sunt dirijate prin adnotări către metode care le procesează, la fel ca

în cazul cererilor HTTP obișnuite. Acest modul conține suport pentru SockJS: un mecanism care imită comportamentul WebSocket prin alte mijloace atunci când browserul sau rețeaua nu permit o conexiune WebSocket directă. Astfel, comunicarea în timp real funcționează chiar și în medii mai restrictive, fără aceste situații să fie tratate separat.

4.2.4 Spring Data JPA

Accesul la o bază de date relațională dintr-o aplicație presupune în mod obișnuit scrierea de interogări SQL și transformarea manuală a rezultatelor în obiecte. Această muncă este repetitivă și predispusă la erori. Pentru a o evita, se folosește o tehnică numită maparea obiect-relațional (în engleză: Object-Relational Mapping sau ORM), prin care tabelele din baza de date sunt legate direct de clase din cod, iar rândurile devin obiecte.

JPA (Java Persistence API) este standardul Java care descrie cum se face această mapare iar Spring Data JPA este modulul Spring care implementează acest standard și simplifică și mai mult lucrul cu baza de date [41]. Ideea principală este cea a depozitelor: se declară o interfață cu metodele de care este nevoie iar Spring generează automat implementarea acestora. De asemenea, multe interogări simple pot fi deduse direct din numele metodei, fără a scrie deloc SQL. Astfel, o mare parte din codul de acces la baza de date dispare, iar logica de stocare și de citire rămâne scurtă și clară. Spring Data JPA folosește de obicei biblioteca Hibernate, care este cea mai cunoscută implementare a standardului JPA.

4.3 Librăria ZXing



ZXing (Zebra Crossing) este o bibliotecă cu sursă deschisă, scrisă în Java, folosită pentru generarea și citirea codurilor de bare, inclusiv a codurilor QR. Este una dintre cele mai cunoscute biblioteci din acest domeniu și este folosită des în aplicațiile care au nevoie să lucreze cu astfel de coduri.

Generarea codurilor se face pornind de la un text, de obicei o adresă URL. ZXing produce matricea de module negre și albe care formează codul QR sau lista de module care formează codul de bare. Acestea se formează urmând standardul ISO/IEC 18004 [4]. Aceste structuri pot fi apoi transformate într-o imagine simplă. Biblioteca se ocupă singură de detaliile codificării, cum ar fi alegerea nivelului de corecție a erorilor și așezarea tiparelor de detecție a poziției, astfel încât aceasta poate fi folosită fără să se cunoască funcționarea internă a standardului. [42]

4.4 Platforma Docker



Docker este platforma aleasă pentru containerizarea și rularea componentelor acestei aplicații datorită eficienței și simplității de utilizare.

Un concept de bază al Docker este construirea imaginilor. O imagine este formată din mai multe straturi suprapuse, fiecare corespunzând unui pas din fișierul de construire (Dockerfile). Aceste straturi sunt păstrate într-o memorie temporară a sistemului, astfel încât la reconstruire se refac doar straturile care s-au schimbat, ceea ce scurtează mult timpul de construire a imaginilor. De asemenea, se pot face construcții în mai multe etape, în care uneltele necesare doar la compilare rămân într-o etapă separată și nu ajung în imaginea finală. În acest fel, imaginea finală rămâne mică și conține doar ce este necesar pentru rulare [25].

Orice imagine pornește de la o imagine de bază declarată în prima instrucțiune a fișierului de construire. Aceasta este primul strat de bază la care se adaugă apoi straturile proprii ale aplicației. O imagine de bază poate fi un sistem de operare redus la strictul necesar sau o imagine care vine deja cu un mediu de rulare pregătit, cum ar fi cel pentru un anumit limbaj de programare. Astfel, nu mai trebuie instalate de la zero toate uneltele de care are nevoie aplicația, fiindcă o parte din ele se moștenesc direct din imaginea de bază.

Alegerea imaginii de bază are urmări asupra dimensiunii finale și a securității. O imagine de bază mai mică înseamnă o imagine finală mai ușor de transferat și de pornit, dar și o suprafață de atac mai redusă, fiindcă pe sistem se află mai puține programe care ar putea avea vulnerabilități. Astfel predomină variantele minimale care conțin doar componentele esențiale. De asemenea, imaginile de bază sunt versionate, ceea ce ajută ca o construcție să dea același rezultat de fiecare dată.

4.5 Baza de date PostgreSQL



PostgreSQL este sistemul de gestiune a bazelor de date ales pentru această lucrare.

Un aspect important al sistemului PostgreSQL este modul în care acesta tratează accesul concurent la date. Se folosește un mecanism de control al concurenței prin mai multe versiuni. Ideea este că, în loc să blocheze datele atunci când cineva le citește, sistemul păstrează mai multe versiuni ale aceluiași rând. Astfel, o operație de citire nu este nevoită să aștepte după o operație

de scriere care se desfășoară în același timp, fiindcă fiecare vede o versiune consistentă a datelor. Acest mecanism, împreună cu suportul pentru tranzații ACID, face ca datele să rămână corecte chiar și sub un trafic ridicat, fără ca performanța să scadă prea mult [29].

4.6 RabbitMQ



RabbitMQ este brokerul de mesaje ales pentru procesarea asincronă a cererilor făcute de clienți către server.

Potrivit [43], RabbitMQ nu trimite mesajele direct în cozi, ci printr-un element intermediar numit schimbător (în engleză: *exchange*). Producătorul publică mesajul către un astfel de schimbător, iar acesta îl dirijează către una sau mai multe cozi, pe baza unor reguli și a unei chei de dirijare atașate mesajului. Acest nivel suplimentar oferă flexibilitate în felul în care mesajele ajung la consumatori. Un alt mecanism important este confirmarea mesajelor: un mesaj este șters din coadă doar după ce consumatorul a confirmat că l-a procesat cu succes, iar dacă procesarea eșuează, mesajul poate fi reintrodus în coadă. De asemenea, cozile pot fi declarate durabile, iar mesajele pot fi marcate ca persistente, ceea ce înseamnă că supraviețuiesc unei reporniri a serverului. Aceste mecanisme împreună fac ca mesajele să nu se piardă în timpul rulării aplicațiilor.

4.7 Serviciul Nginx



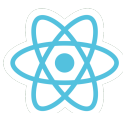
Nginx este un server web cu sursă deschisă cunoscut pentru performanța lui ridicată și pentru consumul redus de resurse. A fost creat inițial pentru a face față unui număr foarte mare de conexiuni simultane. Acest server este folosit ca imagine de bază pentru imaginea Docker a interfeței de utilizator.

Conform [44], în cadrul unei aplicații web, Nginx are două roluri importante:

- Server de fișiere statice: poate livra direct către client paginile HTML, fișierele CSS și JavaScript ale interfeței, fără a mai implica restul aplicației
- Proxy invers: Nginx stă în fața serverului aplicației și primește toate cererile venite de la clienți, apoi le trimite mai departe către componenta potrivită.

Astfel, clientul comunică doar cu Nginx, fără să știe cum este organizată aplicația în detaliu. Acest mod de lucru ascunde structura internă a sistemului și oferă un singur punct simplu de intrare către toate componentele.

4.8 Biblioteca React



React este biblioteca aleasă pentru construirea interfeței de utilizator a aplicației datorită performanței și flexibilității ridicate a acesteia.

Un concept important al bibliotecii React este DOM-ul virtual (Document Object Model). În loc să modifice direct structura paginii la fiecare schimbare, lucru care ar consuma multe resurse, React păstrează o reprezentare a interfeței în memorie. Atunci când datele se schimbă, compară noua reprezentare cu cea veche și actualizează în pagina reală doar acele elemente care s-au modificat efectiv. În acest fel actualizările sunt rapide, iar în aplicație se descrie doar cum trebuie să arate interfața pentru o anumită stare a datelor, fără a se trata modificarea manuală a elementelor. Din acest motiv React se folosește de obicei la construirea aplicațiilor de tip SPA, unde conținutul se actualizează dinamic fără reîncărcarea completă a pagini [10].

Capitolul 5. Proiectare de Detaliu și Implementare

5.1 Detalii de proiectare

5.1.1 Cazuri de utilizare

Cazurile de utilizare descriu care sunt posibilitățile fiecărui utilizator în aplicație. Aplicația are trei tipuri de utilizatori: administrator, profesor și student. Fiecare are propriul set de acțiuni.

Pe lângă acțiunile specifice fiecărui rol, există două acțiuni comune pentru toți utilizatorii. Prima este autentificarea, prin care un utilizator deja existent intră în cont. A doua este înregistrarea, prin care se creează un cont nou. Conturile noi făcute prin înregistrare pot să fie de tip profesor sau student. Contul de administrator se poate crea doar prin instrucțiuni directe către baza de date pentru a restricționa accesul.

Cazuri de utilizare pentru administrator

Administratorul are control asupra datelor din baza de date. Acesta poate să vizualizeze toate tabelele, să editeze entitățile existente și să le șteargă sau să adauge entități noi, cu unele excepții: sistemul nu dă utilizatorului de tip admin control total asupra bazei de date. Astfel, administratorul nu poate să adauge utilizatori și nici să vadă parolele sau tokenurile de autentificare a utilizatorilor.

Adăugarea utilizatorilor se face strict prin metoda de înregistrare descrisă mai sus. Utilizatorul are nevoie de o parolă, iar aceasta se criptează la înregistrare. Scrierea manuală a parolei în baza de date nu respectă confidențialitatea utilizatorilor: adminul are acces la parolă.

În figura 5.1 este ilustrată diagrama de cazuri de utilizare pentru administrator.

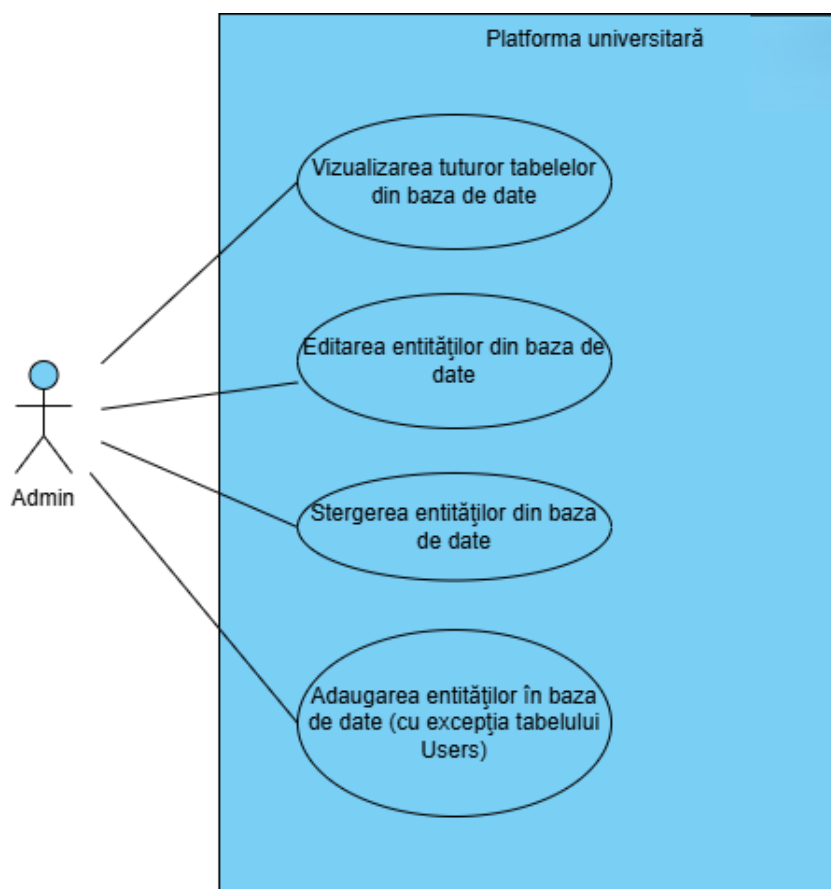


Figura 5.1: Cazuri de utilizare pentru administrator

Cazuri de utilizare pentru profesor

Profesorul este rolul cu cele mai multe acțiuni în aplicație. Acestea implică în principal următoarele:

- controlul sesiunilor de curs și a valabilității codului QR
- vizualizarea în timp real a studenților care se înregistrează
- vizualizarea detaliilor despre cursurile din trecut
- vizualizarea detaliilor despre studenți
- control general asupra notelor studenților

În figura 5.2 se observă toate cazurile de utilizare pentru un utilizator de tip profesor. Unele cazuri de utilizare sunt implicite pentru altele și sunt marcate cu linie punctată.



Figura 5.2: Cazuri de utilizare pentru profesor

Cazuri de utilizare pentru studenți

Studentul are un set mai restrâns de acțiuni, axat pe înregistrarea prezenței și pe vizualizare. În figura 5.3 se prezintă diagrama acestor cazuri de utilizare.

Și la cazurile de utilizare pentru student apare un caz implicit: atunci când studentul își înregistrează prezența, aceasta se trimite automat către profesor, astfel încât pagina profesorului se actualizează în timp real.

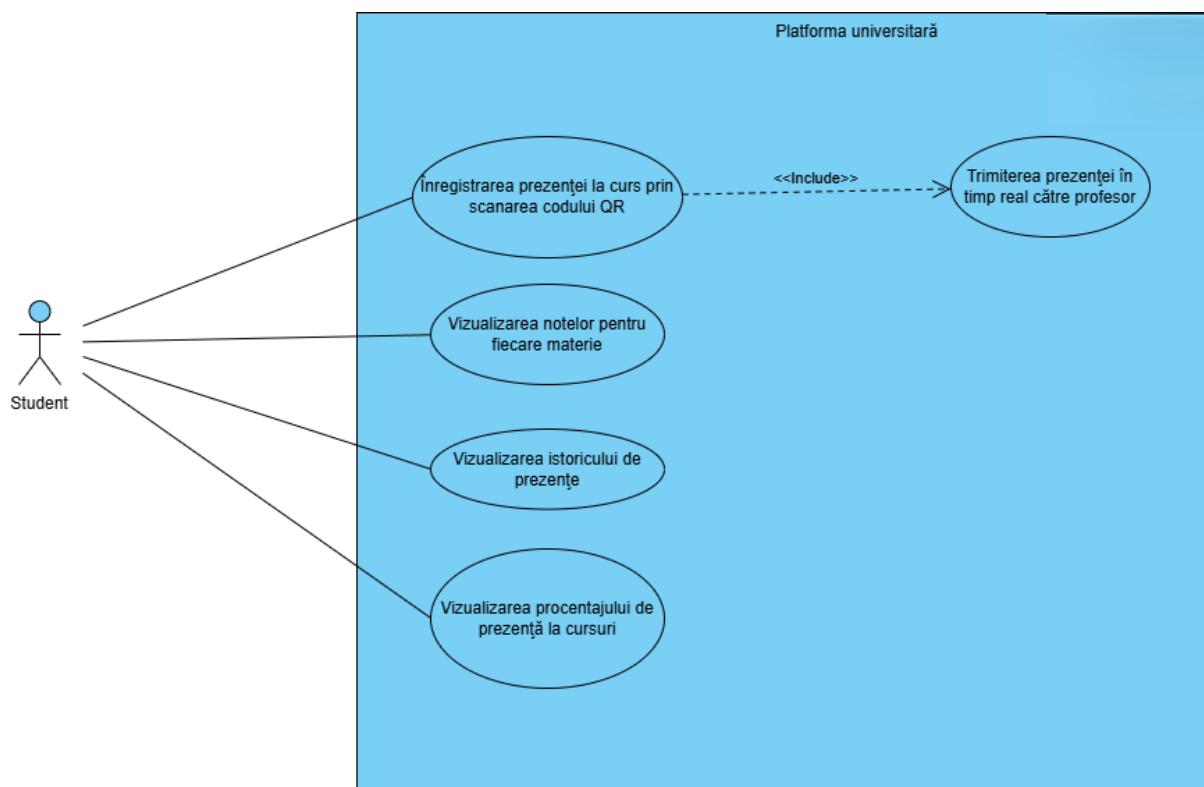


Figura 5.3: Cazuri de utilizare pentru student

5.1.2 Diagrame de secvență

Diagramele de secvență arată cum comunică componentele aplicației între ele și care este ordinea comunicării pentru o anumită acțiune. Se folosesc pentru a înțelege ordinea pașilor și transmiterea datelor. În continuare se descriu trei acțiuni importante: autentificarea, pornirea unei sesiuni de curs și înregistrarea unei prezențe.

Diagrama de secvență pentru autentificare

În figura 5.4 se prezintă procesul de autentificare. Acțiunea pornește de la utilizator, care introduce emailul și parola și apasă butonul "Login". Interfața de utilizator trimite apoi o cerere de tip "POST" către server, la adresa "/api/auth/login" cu emailul și parola.

Serverul caută utilizatorul în baza de date prin "findByEmail". Dacă utilizatorul este găsit, se compară parola din cerere cu parola stocată folosind algoritmul BCrypt. Dacă parolele se potrivesc, serverul generează un token de tip UUID și salvează noua stare a utilizatorului în baza de date.

După salvare, serverul trimite înapoi interfeței de utilizator tokenul, id-ul, numele, emailul și rolul utilizatorului. Interfața salvează tokenul și rolul în "localStorage", memoria browserului. Ultimul pas este redirecționarea utilizatorului către pagina principală corespunzătoare rolului său.

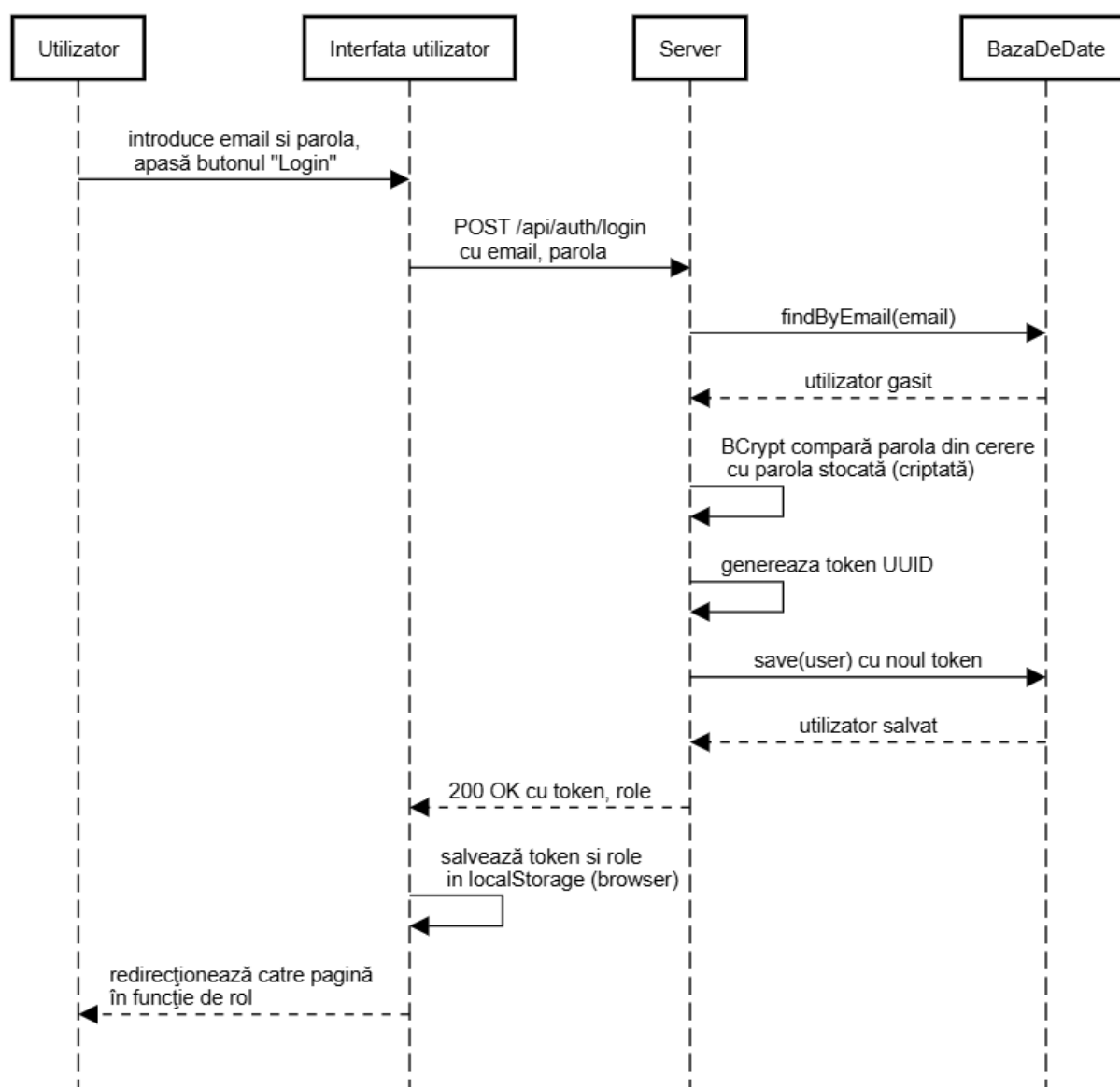


Figura 5.4: Diagrama de secvență pentru autentificare

Diagrama de secvență pentru pornirea unei sesiuni de curs

În figura 5.5 este ilustrată diagrama de secvență pentru pornirea unei sesiuni de curs. Profesorul alege cursul, setează durata de expirare a codului QR și apasă butonul "Start session". Interfața de utilizator ia mai întâi tokenul de autentificare din browser pentru a autoriza cererea.

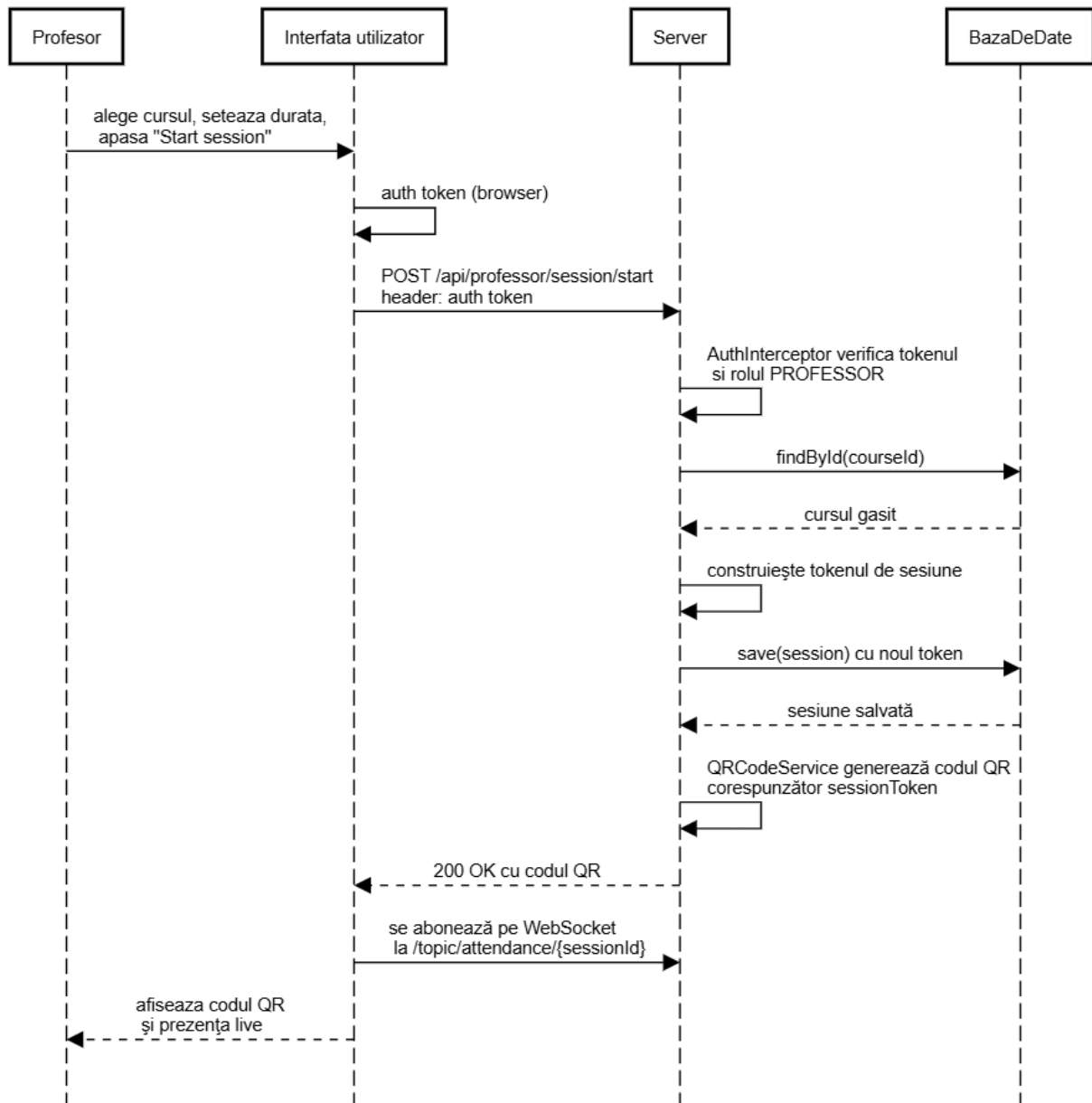


Figura 5.5: Diagrama de secvență pentru pornirea unei sesiuni de curs

Se trimite apoi o cerere "POST" la "/api/professor/session/start" cu tokenul pus în antet (în engleză: header), și id-ul cursului și durata de expirare ca parametrii. Pe server, "AuthInterceptor"

verifică tokenul și confirmă că utilizatorul are rolul de profesor. Dacă verificarea trece, serverul caută cursul prin "findById". Găsirea cursului în baza de date se construiește tokenul de sesiune și se salvează noua sesiune în baza de date.

După ce sesiunea este salvată, instanța de tip "QRCodeService" generează codul QR corespunzător tokenului de sesiune iar serverul trimite codul QR interfeței de utilizator. La final, interfața de utilizator se abonează pe WebSocket la "/topic/attendance/{sessionId}", ca să primească actualizări despre prezența studenților. Astfel, profesorul vede codul QR și lista de prezențe care se actualizează pe măsură ce studenții se înregistrează.

Diagrama de secvență pentru înregistrarea unui prezențe

Înregistrarea unei prezențe este procedura cea mai complexă, pentru că folosește o coadă de mesaje și procesare asincronă. Figura 5.6 prezintă acest proces. Studentul scanează codul QR care îl duce către pagina de înregistrare. Interfața de utilizator ia tokenul de autentificare din browser și trimite o cerere de validare la server cu tokenul de sesiune și tokenul de autentificare.

Cererea către server este interceptată de o instanță de tip "AuthInterceptor" care validează tokenul de autentificare și rolul contului care a făcut cererea, în acest caz rolul de student.

După interceptare, cererea ajunge la "StudentController" care caută sesiunea prin metoda "findBySessionToken" folosind tokenul primit prin cerere. Dacă sesiunea este găsită, se verifică dacă este activă și dacă nu a expirat. În caz afirmativ, serverul trimite înapoi un mesaj de succes iar studentului i se arată pagina de confirmare.

Este nevoie de această primă cerere de verificare către server pentru a preveni trimiterea de cereri de înregistrare a prezenței care nu sunt valide. În cazul în care sesiunea este invalidată de către profesor sau expirată, studentul primește un mesaj de eroare și nu mai poate să trimită cererea de înregistrare a prezenței.

După ce studentul confirmă, se trimite o nouă cerere "POST" la "/api/student/checkin". Serverul verifică din nou tokenul și rolul de student, adaugă detalii despre utilizator pe baza tokenului de autentificare, apoi pune cererea în coada "attendance.queue" din RabbitMQ. Scrierea nu se face încă în baza de date, dar serverul răspunde cu succes, iar studentul primește mesajul de înregistrare reușită.

Restul procesării se întâmplă asincron, în afara fluxului de execuție al cererilor HTTP. Un consumator de tip "RabbitListener" preia mesajul din coadă, face o verificare a cererii astfel încât să nu existe două prezențe pentru aceeași sesiune și același student, după care salvează prezența în baza de date. La final, se trimite o actualizare prin WebSocket către "/topic/attendance/{sessionId}" iar pagina profesorului, care ascultă acest topic, afișează noua prezență în timp real. Această separare între răspunsul rapid și scrierea propriu-zisă face ca aplicația să rămână rapidă chiar dacă mulți studenți se înregistrează în același timp.

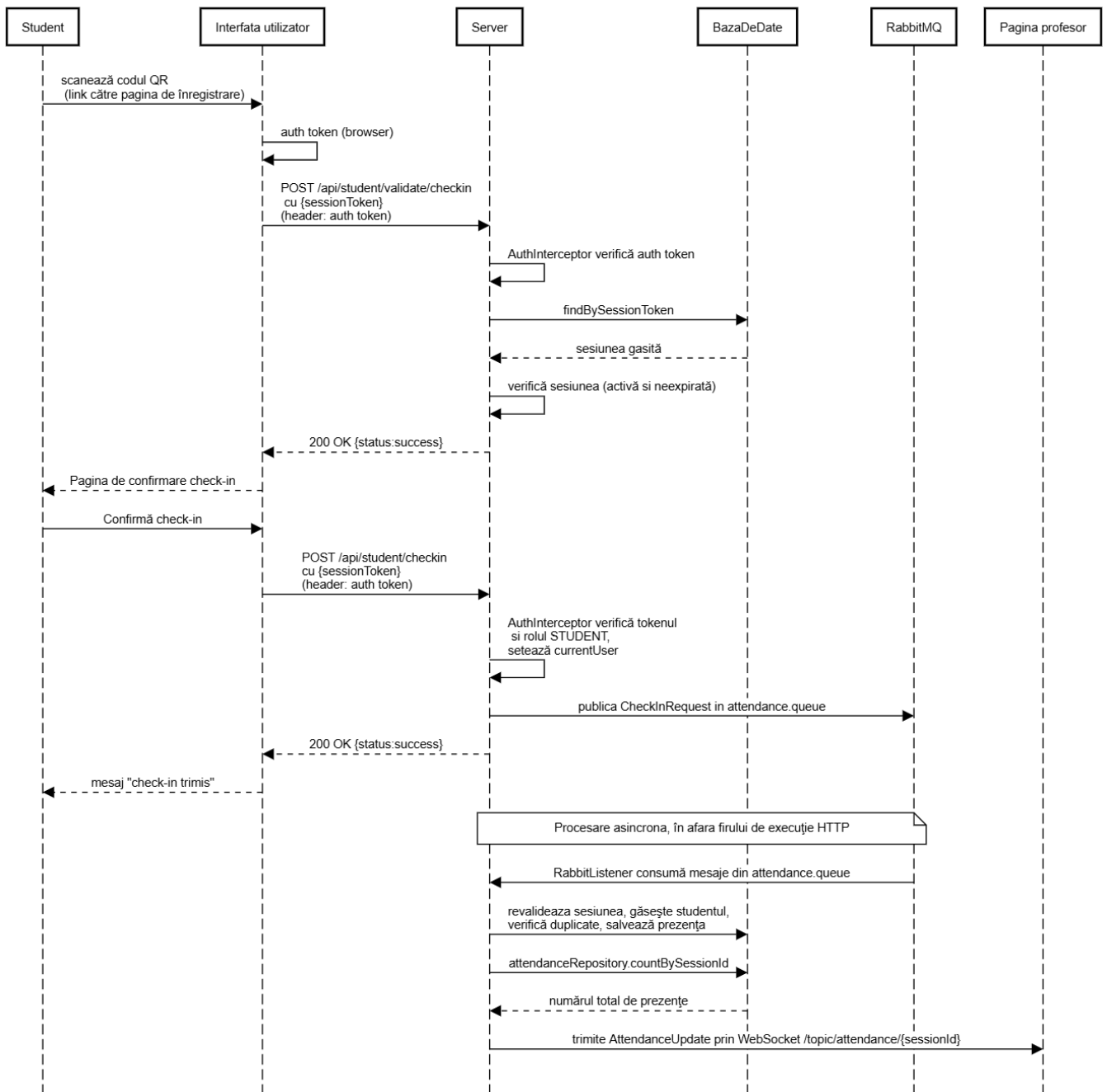


Figura 5.6: Diagrama de secvență pentru înregistrarea unei prezențe

5.1.3 Diagrama de sistem

Diagrama de sistem oferă o vedere de ansamblu asupra componentelor aplicației și asupra modului în care comunică între ele. Figura 5.7 ilustrează această imagine de ansamblu pentru sistemul aplicației care rulează pe calculatorul local. În partea stângă sunt clienții aplicației,

reprezențați prin browserul profesorului și browserul studentului. În partea dreaptă, într-o rețea Docker, se află restul componentelor: interfața de utilizator, serverul SpringBoot, coada de mesaje RabbitMQ și baza de date PostgreSQL.

Comunicarea între componente se face prin mai multe protocoale. Browserele vorbesc cu interfața de utilizator prin HTTP, prin singurul port deschis către calculatorul local din rețeaua Docker. Interfața, la rândul ei, comunică cu serverul tot prin HTTP pentru cererile obișnuite. Serverul folosește protocolul AMQP pentru a vorbi cu RabbitMQ și protocolul JDBC pentru a accesa baza de date.

Conexiunea WebSocket se stabilește inițial printr-o cerere HTTP de la interfața de utilizator către server. După ce conexiunea este stabilită, datele circulă prin protocolul WebSocket într-o singură direcție, de la server către interfață de utilizator. La fel se întâmplă și cu baza de date: protocolul JDBC se inițializează printr-o cerere a serverului către PostgreSQL, însă după inițializare, datele circulă în ambele direcții.

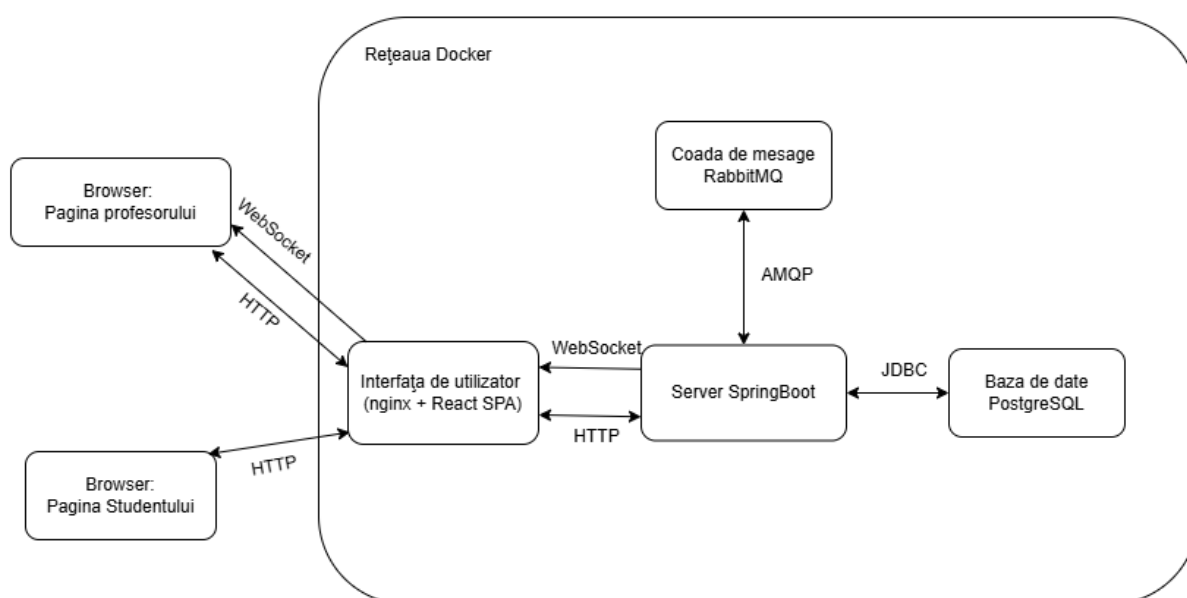


Figura 5.7: Diagrama de sistem a aplicației

5.1.4 Arhitectura bazei de date

Baza de date PostgreSQL are cinci tabele legate între ele prin chei străine (în engleză: foreign key). Figura 5.8 reprezintă schema relațională completă, cu câmpurile fiecărui tabel și relațiile dintre ele.

Tabelul "users" stochează date despre utilizatorii aplicației, indiferent de rol. Câmpurile sale sunt:

- "id" (cheie primară)

- "name": numele utilizatorului
- "email": adresa de email, trebuie să fie unică pentru că se folosește la autentificare
- "password_hash": parola criptată
- "role": rolul utilizatorului (admin, profesor sau student);
- "token": tokenul de autentificare generat la login
- "created_at": data creării entității

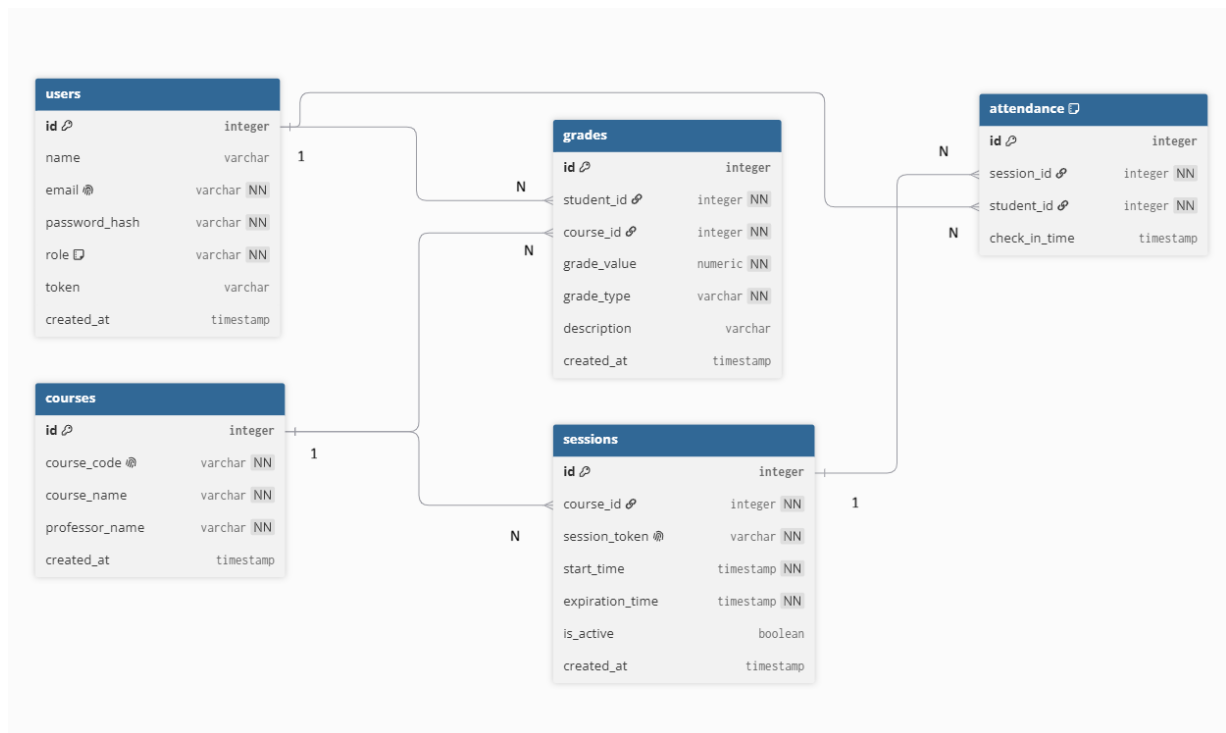


Figura 5.8: Arhitectura bazei de date

Tabelul "courses" stochează date despre cursuri. Acesta conține următoarele câmpuri:

- "id" (cheie primară)
- "course_code": codul cursului, identificator unic în cadrul universității
- "course_name": numele cursului
- "professor_name": numele profesorului, nu este referențiat la câmpul "name" de la tabelul "users"

- "created_at": data creării entității

Structura actuală a aplicației permite tuturor profesorilor să controleze notele pentru orice cursuri, nu există conceptul de custodie a cursurilor. Astfel, în tabelul "courses" nu există câmpuri referențiate la tabelul "users", permițând existența unui curs fără ca profesorul care ține cursul să fie înregistrat pe platformă.

Tabelul "grades" stochează informații despre notele studenților. Câmpurile acestui tabel sunt următoarele:

- "id" (cheie primară)
- "student_id" (cheie străină către "id" din tabelul "users"): studentul căruia îi aparține nota
- "course_id" (cheie străină către "id" din tabelul "courses"): cursul pentru care s-a dat nota
- "grade_value": valoarea numerică a notei
- "grade_type": tipul notei (examen, tema, proiect sau quiz)
- "description": o descriere opțională;
- "created_at": data creării entității

Tabelul "sessions" stochează informații despre sesiunile de curs. Acesta are următoarele câmpuri:

- "id" (cheie primară)
- "course_id" (cheie străină către "id" din tabelul "courses"), cursul pentru care s-a deschis sesiunea
- "session_token": tokenul unic al sesiunii, folosit la generarea codului QR
- "start_time": momentul de pornire a sesiunii de curs
- "expiration_time": momentul de expirare a codului QR
- "is_active": starea sesiunii (activă sau marcată inactivă de către profesor)
- "created_at": data creării entității

Ultimul tabel, "attendance", înregistrează prezența studenților la sesiunile de curs. Câmpurile pentru acest tabel sunt următoarele:

- "id" (cheie primară)
- "session_id" (cheie străină către "id" din tabelul "sessions"): sesiunea de curs la care s-a făcut prezența

- "student_id" (cheie străină către "id" din tabelul "users"): studentul care a făcut prezența
- "check_in_time": momentul înregistrării prezenței

Se observă că un utilizator poate avea mai multe note și mai multe prezențe, un curs poate avea mai multe note și mai multe sesiuni, iar o sesiune poate avea mai multe prezențe.

5.1.5 Structura aplicației

Aplicația este organizată într-un singur depozit (în engleză: repository), care conține atât clientul și serverul cât și infrastructura. Figura 5.9 arată structura folderelor aplicației. Se observă folderul "backend" care conține serverul SpringBoot, și folderul "frontend" care conține aplicația React. Pe lângă acestea există folderul "database" cu fișierul de inițializare a bazei de date și folderul "docs" cu documentația.

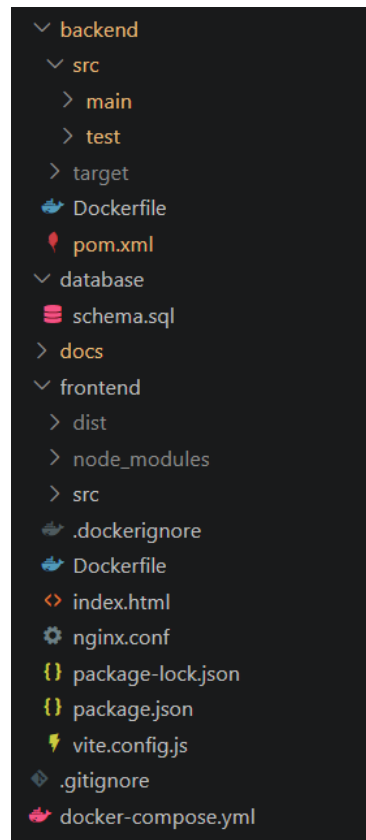


Figura 5.9: Structura folderelor aplicației

În folderul de bază al aplicației se mai găsește fișierul "docker-compose.yml" care pornește toate containerele aplicației și fișierul ".gitignore" care ajută la înlăturarea fișierelor irelevante pentru versionare din sistemul Git. Mai multe detalii despre fișierele aplicației se găsesc în secțiunile de implementare (Vezi 5.2).

5.1.6 Arhitectura serverului SpringBoot

Diagrama de sistem a serverului

Serverul urmărește o arhitectură pe straturi tipică pentru aplicațiile SpringBoot. Cererile de la client ajunge întâi la "AuthInterceptor", care se ocupă de autorizare. De acolo, cererile trec la instanțe de tip "controler" ("Controllers"), apoi la instanțe de tip "serviciu" ("Services"), iar la final la instanțe de tip "depozit" ("Repositories"), care comunică cu baza de date. Figura 5.10 prezintă generic această transmisie de date printr-o diagramă de sistem. Aceasta este de asemenea și reprezentarea detaliată a blocului "Server SpringBoot" din diagrama 5.7.

Stratul "Controllers" publică mesaje pe topicurile RabbitMQ iar stratul "Services" consumă mesaje, ceea ce susține procesarea asincronă. De asemenea, stratul "Services" publică mesaje prin WebSocket pentru actualizările în timp real.

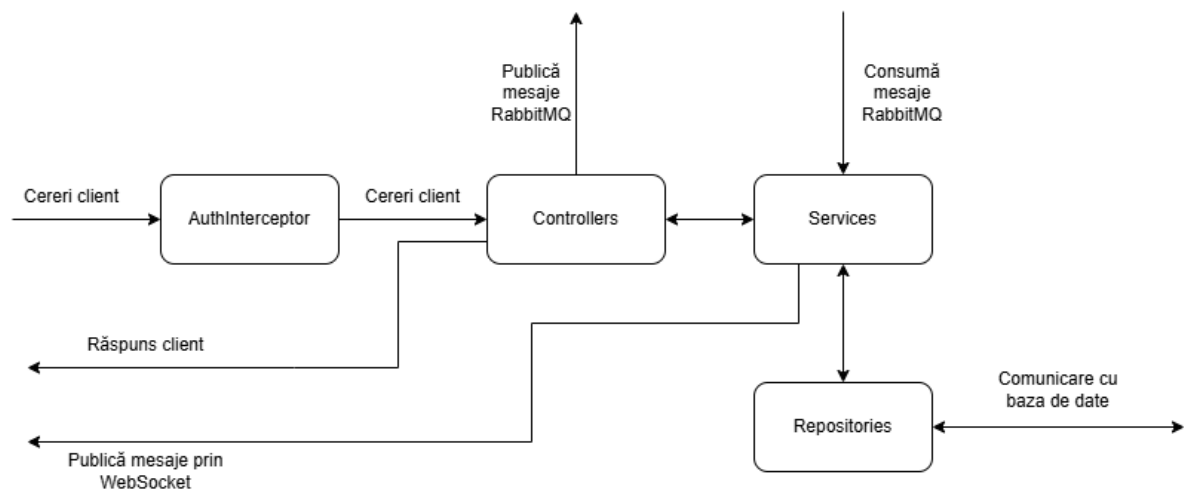


Figura 5.10: Diagrama de sistem a serverului SpringBoot

Structura pachetelor

Codul serverului este împărțit în pachete cu fișiere de același tip. Figura 5.11 arată această structură: în pachetul "com.attendance" se regăsesc pachetele "config", "controller", "dto", "model", "repository" și "service". Acestea reprezintă cele trei straturi și încă trei pachete cu clase pentru modelarea datelor și configurări. Tot aici se află clasa principală "AttendanceSystemApplication.java" și fișierul "application.properties" cu setările aplicației. Detalii relevante despre aceste fișiere și pachete se află în secțiunea de implementare (Vezi 5.2)

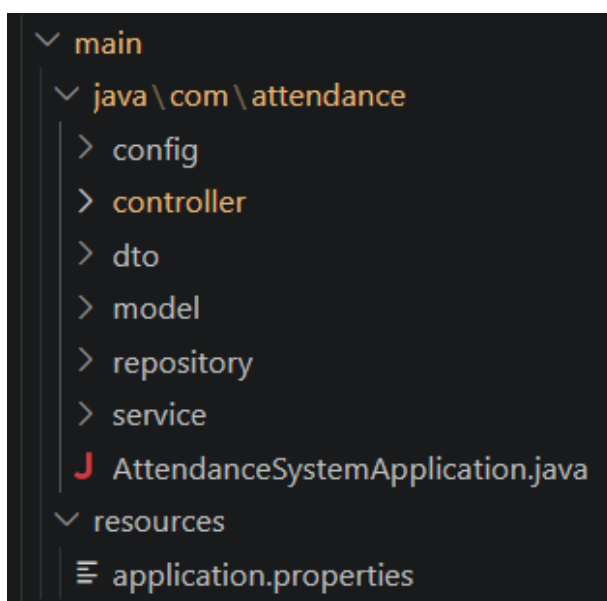


Figura 5.11: Structura pachetelor serverului SpringBoot

Diagrame de clase

Diagramele de clase arată cum se leagă clasele concrete din server, urmând straturile din diagrama de sistem prezentată mai sus. În continuare se prezintă diagrama claselor pentru două acțiuni: pornirea unei sesiuni de curs și înregistrarea unei prezențe.

Figura 5.12 reprezintă clasele implicate în pornirea unei sesiuni de curs. Se observă cum "ProfessorController" folosește "SessionService", iar acesta din urmă folosește "CourseRepository", "SessionRepository" și "QRCodeService". "SessionService" construiește un obiect "Session", îl salvează și returnează un obiect "SessionResponse" care conține și codul QR. Se vede astfel același traseu de la "Controller" la "Service" și apoi la "Repository" descris în diagrama de sistem.

Figura 5.13 ilustrează clasele implicate în înregistrarea unei prezențe pentru o sesiune de curs. În acest caz "StudentController" folosește "AttendanceService", care la rândul lui folosește "SessionRepository" și "AttendanceRepository". "AttendanceService" primește un obiect "CheckInRequest" iar după procesare salvează un obiect "Attendance" și trimite un obiect "AttendanceUpdate" prin WebSocket. Se observă prezența clasei "RabbitTemplate" în componența clasei "AttendanceService" care se folosește pentru a pune cererea în coadă. De asemenea mai există și "SimpMessagingTemplate" care se folosește pentru actualizarea datelor prin WebSocket.

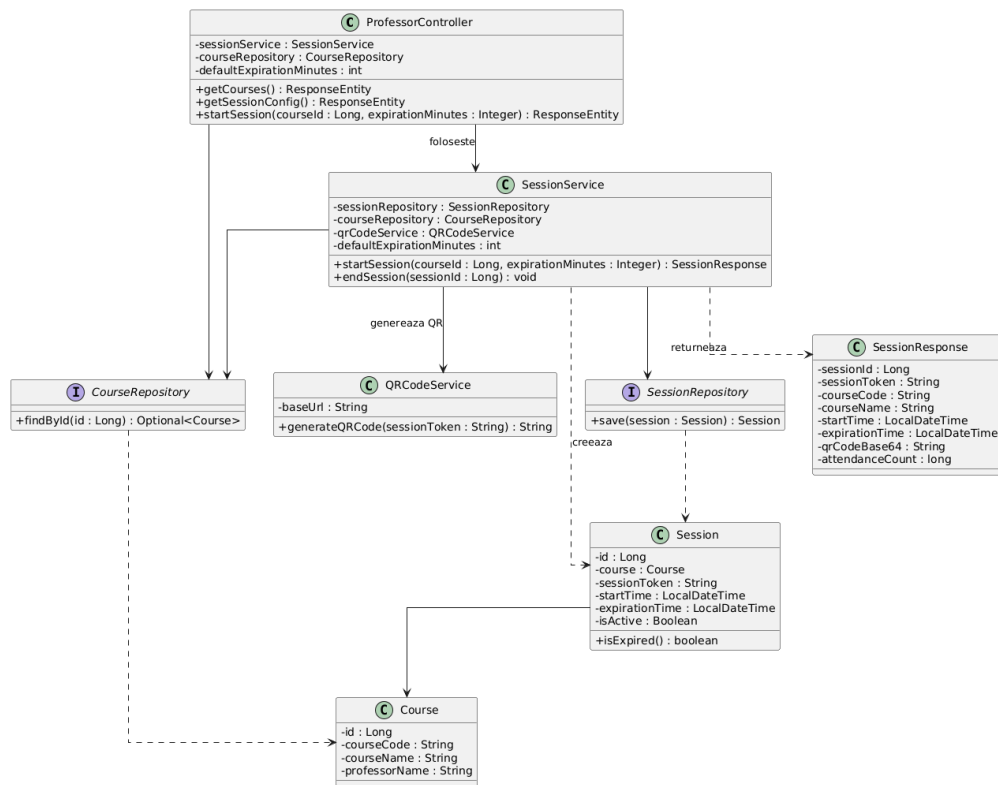


Figura 5.12: Diagrama de clase pentru pornirea unei sesiuni de curs

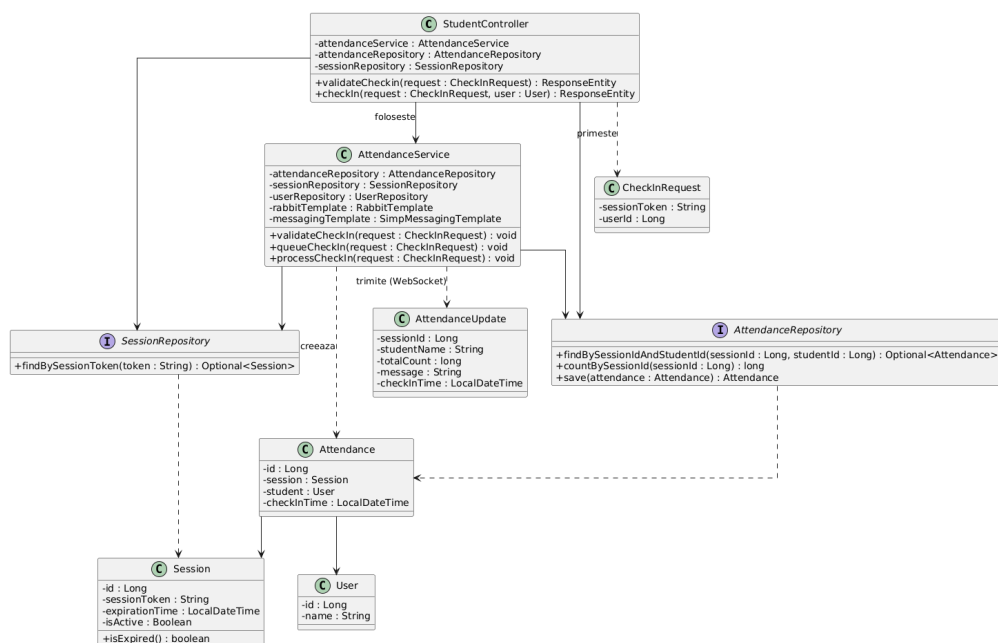


Figura 5.13: Diagrama de clase pentru înregistrarea unei prezențe

5.2 Implementarea serverului SpringBoot

5.2.1 Dependențele aplicației

Toate dependențele aplicației se declară în fișierul "pom.xml". În acesta se regăsește declarată versiunea 17 de Java și versiunea 3.2.0 de "spring-boot-starter-parent" care fixează versiunile bibliotecilor Spring, astfel încât nu trebuie scrise manual.

Dependențele aplicației sunt următoarele:

- "spring-boot-starter-web": pachetul web pentru construirea endpointurilor REST
- "spring-boot-starter-data-jpa": pachetul pentru accesarea bazei de date prin JPA
- "spring-security-crypto": pachetul care conține librăria BCrypt
- "postgresql": driverul pentru baza de date PostgreSQL
- "spring-boot-starter-websocket": pachetul pentru protocolul WebSocket
- "spring-boot-starter-amqp": pachetul pentru utilizarea RabbitMQ
- "zxing": biblioteca folosită pentru generarea codurilor QR
- "lombok": biblioteca pentru generarea codului repetitiv
- "springdoc-openapi-starter-webmvc-ui": biblioteca pentru generarea unui API Swagger

Pe lângă acestea, există și câteva dependențe folosite doar pentru testare: "spring-boot-starter-test", "spring-rabbit-test" și biblioteci din grupul "testcontainers" care pornesc containere reale în timpul testelor de integrare (Vezi capitolul 6) .

5.2.2 Setările aplicației

Setările serverului se află în fișierul "application.properties". Aici se configurează portul, conexiunea la baza de date, conexiunea la RabbitMQ și variabile importante. Listarea 5.1 arată conținutul acestui fișier pentru mediul de rulare local. Acestea sunt valorile care trebuie schimbate atunci când se trece la un mediu de producție.

Câmpurile din fișier sunt următoarele:

- "server.port": portul pe care ascultă serverul
- "spring.datasource.url": adresa bazei de date PostgreSQL
- "spring.datasource.username" și "spring.datasource.password": datele de autentificare la baza de date
- "spring.jpa.hibernate.ddl-auto": se utilizează valoarea "update" astfel încât Hibernate actualizează automat schema bazei de date pe baza entităților

- "spring.jpa.show-sql": afișează în consolă interogările SQL generate
- "spring.jpa.properties.hibernate.dialect": setează dialectul SQL folosit
- "spring.rabbitmq.host", "spring.rabbitmq.port", "spring.rabbitmq.username", "spring.rabbitmq.password": datele de conectare la coada de mesaje RabbitMQ
- "app.qr.default-expiration-minutes": durata implicită de valabilitate a codului QR
- "app.base-url": adresa folosită la construirea linkului din codul QR
- "logging.level.com.attendance": nivelul de logging al aplicației

```
1 server.port=8080
2
3 spring.datasource.url=jdbc:postgresql://localhost:5432/attendance_db
4 spring.datasource.username=postgres
5 spring.datasource.password=password
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.show-sql=true
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
   PostgreSQLDialect
9
10 spring.rabbitmq.host=localhost
11 spring.rabbitmq.port=5672
12 spring.rabbitmq.username=guest
13 spring.rabbitmq.password=guest
14
15 app.qr.default-expiration-minutes=10
16 app.base-url=http://localhost:3000
17
18 logging.level.com.attendance=DEBUG
```

Listing 5.1: Conținutul fișierului "application.properties"

5.2.3 Stratul de controlere

Stratul de controlere este stratul care primește cererile de la clienți. Fiecare clasă de tip controler grupează funcționalități de procesare a cererilor pentru un anumit rol. Listarea 5.2 reprezintă definiția clasei "ProfessorController" împreună cu anotările folosite. Aceste anotări se folosesc la majoritatea claselor de tip controler. Fiecare anotare are un rol bine definit:

- "@RestController": marchează clasa drept un controler REST ale cărui metode returnează date în format JSON

- `@RequestMapping`: stabilește prefixul comun al adreselor pentru toate endpointurile din clasă
- `@RequiredArgsConstructor`: anotare Lombok care generează un constructor cu dependențele clasei
- `@CrossOrigin`: permite cererile venite doar de la anumite adrese, valorile actuale reprezintă adresa interfeței de utilizator a aplicației și adresele aplicației Swagger folosite pentru testare

```
1 @RestController
2 @RequestMapping("/api/professor")
3 @RequiredArgsConstructor
4 @CrossOrigin(origins = {"${app.base-url}", "http://localhost:8090", "
   http://localhost:8080"})
5 public class ProfessorController
```

Listing 5.2: Definiția unei clase de tip controler

Un endpoint reprezintă combinația dintre o adresă URL și o metodă HTTP. Clientul accesează un endpoint pentru a cere sau a trimite date. Listarea 5.3 arată definiția unui endpoint din clasa `ProfessorController`.

Anotarea `@PostMapping` corelează metoda cu o cerere de tip `POST` la adresa `/session/start`. Parametrii cererii sunt marcați cu `@RequestParam`: `courseId` este obligatoriu, iar `expirationMinutes` este opțional. În interiorul metodei se apelează serviciul corespunzător iar răspunsul este returnat printr-un obiect `ResponseEntity` care conține datele și codul de stare HTTP.

```
1 @PostMapping("/session/start")
2 public ResponseEntity<SessionResponse> startSession(
3     @RequestParam Long courseId,
4     @RequestParam(required = false) Integer expirationMinutes) {
5     try {
6         SessionResponse response = sessionService.startSession(courseId,
7             expirationMinutes);
8         return ResponseEntity.ok(response);
9     } catch (Exception e) {
10        return ResponseEntity.badRequest().build();
11    }
12 }
```

Listing 5.3: Definiția unui endpoint din clasa de tip controler

Implementarea celorlalte endpointuri este foarte similară ca și structură, diferă în principal logica de procesare și returnare a datelor. În continuare sunt prezentate endpointurile fiecărui controler.

Controlerul de autentificare ("AuthController") nu are un prefix de rol, fiind accesibil tuturor cererilor făcute de client. Acesta are următoarele endpointuri:

- "POST /register": înregistrarea unui cont nou
- "POST /login": autentificarea unui utilizator și generarea tokenului de autentificare
- "POST /logout": stergerea tokenului de autentificare

Controlerul profesorului ("ProfessorController") expune următoarele endpointuri:

- "GET /courses": lista cursurilor
- "GET /session/config": setează durata implicită de expirare a codului QR la construirea interfeței de utilizator
- "POST /session/start": pornește o sesiune de curs și generează codul QR (listarea 5.3)
- "POST /session/{sessionId}/end": oprește o sesiune de curs activă
- "GET /session/history": detaliile unei sesiuni de curs din trecut
- "GET /attendance-rate": ratele de prezență a studenților pentru un curs

Controlerul studentului ("StudentController") conține:

- "POST /validate/checkin": verifică dacă un cod QR este valid
- "POST /checkin": trimite cererea de înregistrare a prezenței
- "GET /attendance/history": istoricul de prezențe al studentului

Notele sunt gestionate de un controler separat ("GradeController") care are prefixe corespunzătoare pentru student și profesor pentru a stabili nivelul de acces în funcție de roluri. Endpointurile acestui controler sunt:

- "POST /professor/grades": adăugarea unei note
- "DELETE /professor/grades/{gradeId}": ștergerea unei note
- "GET /professor/grades": notele tuturor studenților pentru un curs
- "GET /student/grades": notele studentului care folosește aplicația

Controlerul administratorului ("AdminController") oferă acces direct la datele din baza de date. Acesta conține rute pentru fiecare tabel: "admin/users", "admin/students", "admin/courses", "admin/sessions", "admin/attendance" și "admin/grades". Pentru fiecare dintre aceste rute sunt expuse metodele de citire, creare, modificare și ștergere ("GET", "POST", "PUT" și "DELETE"), singura excepție fiind metoda de creare pentru tabelul "users". De asemenea, funcțiile de citire și modificare a tabelului "users" exclud câmpurile "password_hash" și "token".

5.2.4 Stratul de servicii

Stratul de servicii conține logica aplicației. Aici se execută procesarea datelor primite de la controlere și operațiile necesare înainte ca datele să ajungă la baza de date. În acest strat se generează codurile QR prin clasa "QRCodeService", se consumă mesajele din coada RabbitMQ prin clasa "AttendanceService" și se trimit actualizările în timp real prin WebSocket. Clasele și metodele din acest strat folosesc câteva anotări importante:

- "@Service": marchează clasa drept un serviciu Spring astfel încât aceasta poate fi injectată în alte clase
- "@RequiredArgsConstructor": generează constructorul cu dependențele clasei
- "@Transactional": pune metoda într-o tranzacție de bază de date, astfel operațiile se execută complet sau deloc
- "@Slf4j": adaugă un obiect de logare în clasă

Anotările specifice pentru autentificare, pentru RabbitMQ și pentru WebSocket se descriu separat în secțiunile următoare.

5.2.5 Stratul de accesare a datelor

Stratul de accesare a datelor este ultimul strat al arhitecturii, cel care comunică cu baza de date. Acesta are două părți: clasele care modelează tabelele, numite entități, și interfețele de tip "Repository" care fac operațiile pe aceste tabele. Listarea 5.4 arată definiția entității "User" care corespunde tabelului "users".

Anotările folosite pentru a defini o entitate sunt următoarele:

- "@Entity": marchează clasa ca o entitate JPA corelată de un tabel din baza de date
- "@Table": specifică numele tabelului corespunzător
- "@Data": generează automat metode comune cum ar fi "getter" și "setter"
- "@NoArgsConstructor": generează un constructor fără argumente
- "@AllArgsConstructor": generează un constructor cu toate argumentele

```
1 @Entity
2 @Table(name = "users")
3 @Data
4 @NoArgsConstructor
5 @AllArgsConstructor
6 public class User
```

Listing 5.4: Definiția unei entități corespunzătoare bazei de date

Listarea 5.5 arată definiția interfeței ”UserRepository”. Aceasta extinde interfața ”JpaRepository” de la care moștenește metode pentru salvare, căutare și ștergere. Anotarea ”@Repository” marchează interfața ca o componentă Spring de acces la date.

Pe lângă metodele moștenite, sunt declarate metode proprii doar prin numele lor. Spring Data construiește automat interogările specifice acestor metode pe baza numelor sugestive. Astfel, funcția ”findByEmail” caută un utilizator după email iar ”findByToken” și ”findByRole” caută după token, respectiv rol.

```
1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> {
3     Optional<User> findByEmail(String email);
4     Optional<User> findByToken(String token);
5     List<User> findByRole(Role role);
6 }
```

Listing 5.5: Definiția unei clase de tip ”Repository”

5.2.6 Autentificarea și autorizarea

Autentificarea aplicației este simplă și nu folosește întreg framework-ul de securitate Spring. La login se generează un token de tip UUID care se salvează în baza de date și este comunicat clientului. Clientul trimite apoi acest token în antetul fiecărei cereri. Verificarea tokenului și a rolului cererilor se face printr-o clasă de interceptare înregistrată în clasa de configurare ”WebConfig”. Listarea 5.6 arată această configurare.

Clasa ”AuthInterceptor” se adaugă în registrul de configurare web Spring și se aplică pentru rutele care încep cu ”/api/admin”, ”/api/professor” și ”/api/student”. Rutele de autentificare nu sunt incluse pentru că ele trebuie să fie accesibile fără token.

```
1 @Configuration
2 @RequiredArgsConstructor
3 public class WebConfig implements WebMvcConfigurer {
4     private final AuthInterceptor authInterceptor;
5
6     @Override
7     public void addInterceptors(InterceptorRegistry registry) {
8         registry.addInterceptor(authInterceptor)
9             .addPathPatterns("/api/admin/**", "/api/professor/**", "/api/
10                 student/**");
11     }
12 }
```

Listing 5.6: Configurarea clasei de interceptare

Clasa ”AuthInterceptor” implementează interfața ”HandlerInterceptor” din Spring și suprascrie metoda moștenită ”preHandle”. Această metodă se execută înainte ca cererea să ajungă

la controler și primește ca argument cererea în sine, care poate fi evaluată și la care se pot face modificări. Metoda caută utilizatorul după tokenul de autentificare și adaugă cererii detalii despre utilizator care sunt apoi folosite de controlere. În cazul în care rolul găsit în baza de date nu corespunde cu adresa din cerere, cererea este respinsă și nu ajunge la controlere.

5.2.7 Configurarea și utilizarea componentei RabbitMQ

Configurarea RabbitMQ se face în clasa "RabbitMQConfig" prezentată în listarea 5.7. Coada de mesaje se definește printr-un obiect de tip "Queue" marcat cu anotarea "@Bean", astfel Spring creează acest obiect la pornirea aplicației. Se dau ca parametrii numele cozii și o valoare booleană care stabilește durabilitatea cozii, dacă mesajele se pierd sau nu la repornirea serverului RabbitMQ.

Obiectul "Jackson2JsonMessageConverter" transformă mesajele în format JSON, iar "RabbitTemplate" este obiectul prin care aplicația trimite mesaje în coadă, configurat să folosească convertorul JSON.

```
1 @Configuration
2 public class RabbitMQConfig {
3     public static final String ATTENDANCE_QUEUE = "attendance.queue";
4
5     @Bean
6     public Queue attendanceQueue() {
7         return new Queue(ATTENDANCE_QUEUE, true);
8     }
9
10    @Bean
11    public Jackson2JsonMessageConverter messageConverter() {
12        return new Jackson2JsonMessageConverter();
13    }
14
15    @Bean
16    public RabbitTemplate rabbitTemplate(ConnectionFactory
17        connectionFactory) {
18        RabbitTemplate template = new RabbitTemplate(connectionFactory);
19        template.setMessageConverter(messageConverter());
20        return template;
21    }
22 }
```

Listing 5.7: Configurarea cozii de mesaje RabbitMQ

Utilizarea cozii se face în clasa "AttendanceService". Atunci când un student trimite o cerere de prezență, metoda "queueCheckIn" pune cererea în coadă prin metoda "convertAndSend" a clasei de tip "RabbitTemplate". Pentru procesarea cererii se folosește metoda "processCheckIn"

a aceluiași serviciu, care este marcată cu anotarea Spring ”@RabbitListener. Această anotare marchează metoda ca și consumator RabbitMQ care preia mesajele din coadă și le procesează.

5.2.8 Configurarea și utilizarea protocolului WebSocket

Configurarea WebSocket se face în clasa ”WebSocketConfig” prezentată în listarea 5.8. Anotarea Spring ”@EnableWebSocketMessageBroker” activează suportul pentru mesaje prin WebSocket folosind protocolul STOMP.

În metoda ”configureMessageBroker” se creează un broker simplu cu prefixul ”/topic” pe care serverul publică actualizările. În metoda ”registerStompEndpoints” se creează endpointul HTTP de stabilire a conexiunii WebSocket ”/ws”. Acest endpoint permite cererile de la interfața de utilizator și localhost, și activează compatibilitatea cu SockJS, o bibliotecă JavaScript care asigură funcționarea prin metode HTTP atunci când browserul nu suportă WebSocket.

```
1 @Configuration
2 @EnableWebSocketMessageBroker
3 public class WebSocketConfig implements
4     WebSocketMessageBrokerConfigurer {
5     @Value("${app.base-url}")
6     private String baseUrl;
7
8     @Override
9     public void configureMessageBroker(MessageBrokerRegistry config) {
10         config.enableSimpleBroker("/topic");
11     }
12
13     @Override
14     public void registerStompEndpoints(StompEndpointRegistry registry)
15     {
16         registry.addEndpoint("/ws")
17             .setAllowedOrigins(baseUrl, "http://localhost:8090", "http://localhost:8080")
18             .withSockJS();
19     }
20 }
```

Listing 5.8: Clasa de configurare WebSocket

Trimiterea actualizărilor prin WebSocket se face în clasa ”AttendanceService”. După ce o prezență la curs este salvată în baza de date, se apelează metoda ”convertAndSend” a clasei ”SimpMessagingTemplate” cu topicul ”/topic/attendance/{sessionId}” și datele despre prezența actualizată. Pagina profesorului, care este abonată la acest topic, primește datele și afișează noua prezență fără a reîncărca pagina.

5.3 Implementarea interfeței de utilizator

5.3.1 Biblioteci externe utilizate

Bibliotecile externe ale interfeței de utilizator se declară în fișierul "package.json", împreună cu comenzile de rulare și de construire a aplicației. Acest fișier separă dependențele folosite la rulare de cele folosite la construirea aplicației.

Dependențele principale, cele folosite pentru rularea aplicației, sunt următoarele:

- "react" și "react-dom": bibliotecile React de bază
- "react-router-dom": extensie React pentru gestionarea rutelor
- "@stomp/stompjs": bibliotecă folosită pentru mesaje prin WebSocket
- "sockjs-client": bibliotecă cu funcționalități alternative WebSocket
- "react-icons": extensie React care conține mici desene

Pe lângă acestea există dependențele de dezvoltare, cele folosite la construirea aplicației:

- "vite": unealta care se folosește la compilarea codului și la rularea locală a aplicației
- "@vitejs/plugin-react": extensie Vite pentru compatibilitatea cu React

În folderul de bază al aplicației se mai găsește și fișierul "package-lock.json", generat automat de managerul de pachete "npm". Acesta conține versiunile exacte ale tuturor dependențelor, astfel încât instalarea să fie exact la fel în orice mediu.

5.3.2 Configurarea serverului

Serverul care livrează paginile aplicației diferă în funcție de mediul în care rulează aplicația: se folosește server Vite pentru rularea locală a aplicației și server Nginx pentru rularea aplicației în containerul Docker. Astfel, există două fișiere de configurare a serverului: "vite.config.js" și "nginx.conf".

În configurarea serverului Vite din fișierul "vite.config.js" se setează portul 3000 pe care ascultă serverul și se definesc redirectionările (proxy) care trimit cererile către "/api" și "/ws" mai departe către portul serverului SpringBoot. În configurația actuală, cererile se direcționează către adresa "localhost:8090" care corespunde cu serverul SpringBoot care rulează în container Docker (rularea locală expune serverul pe portul 8080).

Fișierul de configurare Nginx, listat la 5.9, conține un singur bloc, "server" care configurează serverul să asculte pe portul 80 și să servească fișierele din folderul în care a fost copiată aplicația compilată. În acestui bloc se găsesc trei blocuri de tip "location":

- "location /api/": redirectionează cererile către serverul SpringBoot, accesat prin rețeaua Docker. Se atașează în antetul cererii redirectionate detalii despre cererea originală

- "location /ws/": redirecționează cererea de conexiune WebSocket către server. Sunt necesare antetele "Upgrade" și "Connection" pentru a specifica trecerea de la protocolul HTTP la conexiune WebSocket
- "location /": servește paginile aplicației. Comanda "try_files" face ca, indiferent de adresa cerută, să se returneze o pagină existentă

```
1 server {
2     listen 80;
3     server_name localhost;
4     root /usr/share/nginx/html;
5     index index.html;
6
7     location /api/ {
8         proxy_pass http://app:8080;
9         proxy_set_header Host $host;
10        proxy_set_header X-Real-IP $remote_addr;
11        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
12        proxy_set_header X-Forwarded-Proto $scheme;
13    }
14
15    location /ws/ {
16        proxy_pass http://app:8080;
17        proxy_http_version 1.1;
18        proxy_set_header Upgrade $http_upgrade;
19        proxy_set_header Connection "upgrade";
20        proxy_set_header Host $host;
21        proxy_set_header X-Real-IP $remote_addr;
22        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
23        proxy_read_timeout 86400;
24    }
25
26    location / {
27        try_files $uri $uri/ /index.html;
28    }
29 }
```

Listing 5.9: Conținutul fișierului "nginx.conf"

5.3.3 Paginile aplicației

Interfața de utilizator este împărțită în pagini, fiecare corespunzând unei acțiuni sau unui rol. Paginile se află în folderul "pages" și sunt de tip JSX. În continuare este prezentată descrierea fiecărei pagini:

- "Login.jsx": pagina de autentificare și de înregistrare
- "ProfessorDashboard.jsx": pagina profesorului de unde se pornește o sesiune și se afișează codul QR și lista de prezențe
- "ProfessorHistory.jsx": pagină pentru profesori cu istoricul sesiunilor de curs și detalii despre prezențe
- "ProfessorGrades.jsx": locul de unde se gestionează notele
- "StudentCheckin.jsx": pagina de înregistrare a prezenței, deschisă prin scanarea QR
- "StudentHistory.jsx": istoricul prezențelor studentului
- "StudentGrades.jsx": vizualizarea notelor studentului
- "AdminPage.jsx": panoul de administrare prin care se gestionează datele din baza de date

Pe lângă aceste pagini, fișierul "NavBar.jsx" din folderul "components" definește bara de navigare care este componentă a fiecărei pagini. Aceasta afișează numele utilizatorului, rolul lui și un buton de deconectare. Opțiunile de navigare sunt definite în funcție de rol.

5.3.4 Rutarea paginilor și protejarea rutelor

Rutarea leagă fiecare adresă din browser de o pagină. Se folosește biblioteca "react-router-dom" pentru a defini rutele în fișierul "main.jsx". În listarea 5.10 este prezentată rutarea adresei "/professor/grades" la pagina "ProfessorGrades".

```
1 <Route
2   path="/professor/grades"
3   element={
4     <ProtectedRoute allow={['PROFESSOR']}>
5     <ProfessorGrades />
6     </ProtectedRoute>
7   }
8 />
```

Listing 5.10: Rutarea unei pagini

Accesarea paginilor este restricționată de funcția "ProtectedRoute", prezentată în listarea 5.11. Aceasta primește rolurile permise prin parametrul "allow" și pagina care trebuie protejată și afișată prin parametrul "children".

În interiorul ei se fac două verificări. În primul rând, se verifică dacă utilizatorul are un token, dacă nu are, este trimis la pagina de autentificare. Pagina inițială pentru care s-a făcut cererea este salvată în câmpul "state" iar utilizatorul este retrimis automat către aceasta după autentificare. În al doilea rând, se verifică dacă rolul utilizatorului se află printre rolurile permise: dacă nu, este trimis către pagina lui principală. Doar atunci când ambele verificări trec pagina cerută este afișată.

```
1 export default function ProtectedRoute({ allow, children }) {
2   const auth = getAuth();
3   const location = useLocation();
4
5   if (!auth?.token) {
6     return <Navigate to="/login" state={{ from: location.pathname +
7       location.search }} replace />;
8   }
9   if (allow && !allow.includes(auth.role)) {
10    return <Navigate to={homePathFor(auth.role)} replace />;
11  }
12  return children;
13 }
```

Listing 5.11: Funcția "ProtectedRoute"

5.3.5 Gestionarea sesiunii utilizatorului

După autentificare, tokenul primit de la server, rolul și numele utilizatorului se stochează în "localStorage", memoria browserului. Funcțiile care lucrează cu aceste date se află în fișierul "auth.js" și sunt prezentate la listarea 5.12.

Funcția "getAuth" citește datele din "localStorage" și le transformă din text în obiect. Dacă nu există nimic salvat sau apare o eroare, returnează valoarea nulă. Funcția "setAuth" transformă obiectul în text și îl salvează. Cheia folosită este "auth", definită o singură dată ca o constantă. Mai există și funcția "clearAuth" care șterge aceste date și este apelată atunci când utilizatorul apasă butonul de deconectare.

```
1 const STORAGE_KEY = 'auth';
2
3 export function getAuth() {
4   try {
5     const raw = localStorage.getItem(STORAGE_KEY);
6     return raw ? JSON.parse(raw) : null;
7   } catch {
8     return null;
9   }
10 }
11
12 export function setAuth(auth) {
13   localStorage.setItem(STORAGE_KEY, JSON.stringify(auth));
14 }
```

Listing 5.12: Funcțiile de scriere și citire a sesiunii de utilizator

Toate cererile către server trec printr-o singură funcție, numită ”apiFetch”, prezentată în listarea 5.13. Rolul acestei funcții este de a adauga automat tokenul de autentificare în antetul ”Authorization”, astfel acesta poate sa fie interceptat de către server.

Funcția tratează și cazul în care serverul răspunde cu codul 401 care reprezintă lipsa tokenului. În acest caz, datele de autentificare se șterg, iar utilizatorul este redirecționat către pagina de autentificare.

```
1 export async function apiFetch(url, options = {}) {
2   const auth = getAuth();
3   const headers = { ...(options.headers || {}) };
4   if (auth?.token) headers.Authorization = `Bearer ${auth.token}`;
5
6   const res = await fetch(url, { ...options, headers });
7   if (res.status === 401) {
8     clearAuth();
9     if (window.location.pathname !== '/login') window.location.href =
10       '/login';
11   }
12   return res;
13 }
```

Listing 5.13: Funcția de apelare a endpointurilor

5.3.6 Actualizarea datelor în timp real

Pagina profesorului trebuie să afișeze prezențele pe măsură ce studenții se înregistrează, fără reîncărcare. Acest lucru se face prin funcția ”connectWebSocket” prezentată în listarea 5.14. Funcția se apelează după ce profesorul pornește o sesiune, folosind identificatorul sesiunii primit de la server.

În interiorul funcției se creează un client STOMP printr-o conexiune SockJS către adresa ”/ws” a serverului. Dacă există deja o conexiune veche, aceasta se închide înainte de a începe una nouă. Atunci când conexiunea reușește, clientul se abonează la topicul ”/topic/attendance/” specific identificatorului de sesiune. La fiecare mesaj primit pe acest topic, se actualizează numărul total de prezențe și se adaugă noul student în listă, împreună cu ora înregistrării. Opțiunea ”reconnectDelay” face ca, în cazul deconectării, clientul să încerce reconectarea după câteva secunde.

SockJS imită comportamentul protocolului WebSocket. Rolul acestei librării este de a menține o conexiune între client și server chiar și atunci când browserul sau rețeaua nu permit folosirea WebSocket în mod direct. În aceste situații, SockJS folosește automat alte metode de transport bazate pe HTTP, cum ar fi sondajele lungi (în engleză: long polling), astfel încât conexiunea să rămână funcțională. Se observă în listare că nu se folosește direct un obiect WebSocket, ci apelul ”new SockJS('/ws')”. Astfel, dacă WebSocket nu este disponibil, alegerea metodei alternative se face de către SockJS.

```
1  const connectWebSocket = (sessionId) => {
2  if (stompClientRef.current) {
3      stompClientRef.current.deactivate();
4  }
5  const client = new Client({
6      websocketFactory: () => new SockJS('/ws'),
7      reconnectDelay: 5000,
8      onConnect: () => {
9          client.subscribe('/topic/attendance/' + sessionId, (msg) => {
10             const update = JSON.parse(msg.body);
11             setCount(update.totalCount);
12             setAttendees((prev) => [
13                 {
14                     name: update.studentName,
15                     time: new Date(update.checkInTime).toLocaleTimeString('ro-RO', { timeZone: 'Europe/Bucharest' }),
16                 },
17                 ...prev,
18             ]);
19             });
20         },
21     });
22     client.activate();
23     stompClientRef.current = client;
24 };
```

Listing 5.14: Funcția ”connectWebSocket”

5.4 Implementarea infrastructurii

5.4.1 Containerizarea serverelor

Serverul SpringBoot și interfața pentru utilizator sunt incluse în imagini Docker pentru a putea fi ușor de folosit și portat în alte medii. Fiecare are propriul fișier ”Dockerfile”, care descrie pașii de construire a imaginii. Ambele fișiere folosesc construirea în mai multe etape prin care imaginea finală conține doar ce este necesar pentru rulare, fără uneltele de construire.

Listarea 5.15 reprezintă fișierul ”Dockerfile” al serverului SpringBoot. Prima etapă pornește de la o imagine cu Maven și Java 17. Se copiază fișierul ”pom.xml” și se descarcă dependențele, apoi se copiază codul sursă și se construiește arhiva ”.jar”. A doua etapă pornește de la imaginea ”eclipse-temurin:17-jre-alpine”, o imagine mai mică care conține doar mediul de rulare Java. Aici se copiază fișierele construite în etapa anterioară, se expune portul 8080 și se stabilește comanda de pornire a aplicației care este rulată la pornirea containerului.

```
1 FROM maven:3.9-eclipse-temurin-17 AS build
2
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn dependency:go-offline
6 COPY src ./src
7 RUN mvn clean package -DskipTests
8
9 FROM eclipse-temurin:17-jre-alpine
10
11 WORKDIR /app
12 COPY --from=build /app/target/*.jar app.jar
13 EXPOSE 8080
14
15 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 5.15: Dockerfile pentru serverul SpringBoot

Listarea 5.16 reprezintă fișierul "Dockerfile" al interfeței de utilizator. Prima etapă pornește de la o imagine cu Node.js pentru a putea construi aplicația cu unealta Vite. Se instalează dependențele, se copiază codul și se compilează, rezultând fișiere statice. De asemenea, se stabilește numele universității printr-un argument de construire. A doua etapă folosește o imagine cu serverul web Nginx. În ea se copiază fișierele statice construite și fișierul de configurare "nginx.conf" (Vezi 5.3.2). Ultima comandă din fișier este cea de pornire a serverului Nginx.

```
1 FROM node:20-alpine AS build
2
3 WORKDIR /app
4 COPY package.json ./
5 RUN npm install
6 COPY . .
7 ARG VITE_UNIVERSITY_NAME
8 ENV VITE_UNIVERSITY_NAME=$VITE_UNIVERSITY_NAME
9 RUN npm run build
10
11 FROM nginx:1.27-alpine
12
13 COPY --from=build /app/dist /usr/share/nginx/html
14 COPY nginx.conf /etc/nginx/conf.d/default.conf
15 EXPOSE 80
16
17 CMD ["nginx", "-g", "daemon off;"]
```

Listing 5.16: Dockerfile pentru interfața de utilizator

5.4.2 Crearea rețelei Docker

Toate componentele aplicației pornesc împreună prin utilizarea fișierul "docker-compose.yml". Acesta descrie serviciile, legăturile dintre ele și rețeaua comună. Toate serviciile se află în aceeași rețea, numită "attendance-network", astfel încât pot comunica între ele folosind numele.

Serviciile pentru serverele SpringBoot și Nginx folosesc imaginile construite la punctul anterior. Celelalte servicii folosesc imagini descărcate de Docker din Docker Hub: "postgres:15-alpine" pentru baza de date, "rabbitmq:3-management-alpine" pentru coada de mesaje și "dpage/pgadmin4" pentru administrarea bazei de date.

Baza de date se inițializează automat la prima pornire. Fișierul "schema.sql" se copiază în folderul de inițializare al containerului PostgreSQL, iar acesta îl rulează automat creând tabelele și datele de început. Pentru ca datele să nu se piardă la repornire, se folosește un volum Docker numit "postgres-data" în care containerul PostgreSQL păstrează informațiile pe calculatorul local. Portul deschis către calculatorul local pentru a folosi aplicația este cel al interfeței de utilizator, portul 3000, restul comunicării rămânând în interiorul rețelei Docker. Instrucțiunile exacte de pornire a aplicației sunt prezentate în secțiunea 7.1.

Capitolul 6. Testare și Validare

Toate testele automate se află în folderul "backend/src/test" care urmează aceeași structură de pachete ca și codul sursă. Astfel, testele unitare se află în subfoldere, iar testele de integrare sunt în folderul principal și au sufixul "IT" pentru a putea fi rulate separat de cele unitare. Această separare se face prin două plugin-uri Maven: "surefire" rulează testele unitare, iar "failsafe" rulează testele de integrare.

6.1 Testarea unitară

În testarea unitară se folosește biblioteca "Mockito" pentru a introduce obiecte false în locurile irelevante din cod împreună cu "JUnit 5". Există teste unitare pentru clasele care conțin logica importantă. Acestea sunt următoarele:

- "AuthServiceTest": verifică înregistrarea și autentificarea
- "AttendanceServiceTest": verifică validarea unei prezențe și trimiterea de cereri de prezență
- "SessionServiceTest": verifică funcționalitățile sesiunii de curs
- "GradeServiceTest": verifică regulile de validare a notelor
- "AuthInterceptorTest": verifică filtrul de autorizare
- "SessionTest": verifică expirarea codului QR

Rularea se face cu comanda "mvn test". În listarea 6.1 se observă rezultatul: toate cele 42 de teste trec fără erori în aproximativ patru secunde.

```
1 [INFO] Tests run: 42, Failures: 0, Errors: 0, Skipped: 0
2 [INFO]
3 [INFO] -----
4 [INFO] BUILD SUCCESS
5 [INFO] -----
6 [INFO] Total time: 4.339 s
7 [INFO] Finished at: 2026-06-22T21:09:31+03:00
8 [INFO] -----
```

Listing 6.1: Rezultatul rulării testelor unitare

6.2 Testarea de integrare

Pentru testarea de integrare se folosesc biblioteci din grupul "Testcontainers" care creează containere pentru RabbitMQ și PostgreSQL. Clasa "AbstractIntegrationTest" implementează pornirea celor două containere și configurează serverul SpringBoot să se conecteze la ele. Clasele de test de integrare extind această clasă.

Există trei clase de teste de integrare:

- "AuthFlowIT": verifică procesul de autentificare prin cereri HTTP reale. Tokenul de student ajunge la rutele de student, dar este rejectat pe rutele de admin
- "CheckInFlowIT": verifică procesul de înregistrare a prezenței. Cererea intră în coadă, iar după un scurt timp prezența apare în baza de date
- "CheckInLoadIT": este testul de performanță discutat în secțiunea următoare

Rularea se face cu comanda "mvn verify" care rulează întâi testele unitare și apoi pe cele de integrare. În listarea 6.2 se observă că toate cele 7 teste de integrare trec fără erori. Timpul total este de aproape un minut pentru că se creează containere și se pornește serverul care comunică cu acestea.

```
1 [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
2 [INFO]
3 [INFO]
4 [INFO] --- failsafe:3.1.2:verify (default) @ attendance-system ---
5 [INFO] -----
6 [INFO] BUILD SUCCESS
7 [INFO] -----
8 [INFO] Total time: 51.658 s
9 [INFO] Finished at: 2026-06-22T21:16:12+03:00
10 [INFO] -----
```

Listing 6.2: Rezultatul rulării testelor de integrare

6.3 Testarea de performanță

Testul de performanță simulează momentul în care profesorul afișează codul QR și toți studenții îl scanează aproape în același timp. Clasa "CheckInLoadIT" simulează 300 de studenți care trimit cererea de prezență în paralel. Pentru asta se folosește un grup de fire de execuție (în engleză: thread pool) și o barieră comună, astfel încât toate cererile să pornească în același moment. Se măsoară două valori: timpul firului de execuție HTTP (cât durează până când toți studenții primesc răspuns) și timpul de procesare completă (cât durează până când cererile ajung în baza de date).

Testul s-a rulat în trei configurații diferite pentru a se observa efectul cozii de mesaje asupra aplicației. În continuare se prezintă rezultatele.

Rezultatul testului de performanță cu un singur consumator RabbitMQ:

- HTTP response: 1485 ms
- Full processing: 6836 ms

Rezultatul testului de performanță fără RabbitMQ (procesare sincronă direct în cerere):

- HTTP response: 3414 ms
- Full processing: 3541 ms

Rezultatul testului de performanță cu mai mulți consumatori RabbitMQ:

- HTTP response: 1774 ms
- Full processing: 3859 ms

Din aceste rezultate se observă câteva lucruri. În varianta fără RabbitMQ, timpul de răspuns HTTP și cel de procesare totală sunt aproape egale. Acest lucru este de așteptat pentru metoda de interceptare a cererii HTTP face toată procesarea.

În varianta cu RabbitMQ și un singur consumator, timpul de răspuns al firului de execuție HTTP scade la jumătate față de varianta sincronă. În schimb, timpul de procesare completă crește semnificativ pentru că există un singur fir de execuție care procesează cererile, comparativ cu procesarea paralelă a serverului SpringBoot din cazul anterior. Totuși, acest lucru nu deranjează utilizatorul, fiindcă acesta a primit deja confirmarea iar cererea este deja în sistem.

Varianta cu mai mulți consumatori RabbitMQ oferă clar cea mai bună performanță în timp. Timpul de răspuns HTTP este la jumătate față de varianta fără RabbitMQ iar timpul de procesare totală rămâne la fel, jumătate din timpul de procesare totală a variantei cu un singur consumator. Totuși, creșterea numărului de consumatori vine cu o posibilă problemă: două mesaje repetate ale aceluiași student ar putea fi procesate în același timp de consumatori diferiți, astfel, verificarea de cereri duplicate trece în ambele fire de execuție.

6.4 Testarea interfeței de utilizator

Pe lângă testele automate, s-a făcut și o testare manuală a interfeței de utilizator. Aici scopul a fost să se verifice că fiecare funcționalitate, văzută din ochii utilizatorului, se comportă corect în browser. Pentru asta s-a creat câte un cont pentru fiecare rol: un administrator, un profesor și un student. Pentru fiecare rol s-au testat manual pe rând toate funcționalitățile. În toate cazurile rezultatul obținut a fost cel așteptat.

Interacțiunea dintre profesor și student în cadrul aplicației și scanarea de cod QR au fost simulate cu o captură de ecran a codului QR afișat de profesor. Această captură s-a încărcat într-un cititor de coduri QR care a extras linkul, apoi acesta s-a deschis într-o nouă fereastră de browser privată, ca să simulăm o sesiune separată, diferită de sesiunea profesorului. Astfel, telefonul nu mai trebuie conectat prin rețea la calculator și testarea este mai rapidă.

Capitolul 7. Manual de Instalare și Utilizare

7.1 Manual de instalare

Metoda cea mai simplă și rapidă de instalare este folosind Docker și Docker Compose. Având aceste aplicații instalate pe calculatorul local, aplicația se pornește cu o singură comandă rulată în folderul de bază al aplicației: "docker-compose up". Se accesează apoi interfața de utilizator în browser la adresa "http://localhost:3000".

Pentru pornirea granulară a fiecărei componente, în cazul în care se dorește dezvoltarea sau depănarea aplicației, sunt necesare următoarele:

- "Java 17" și "Maven 3.9": pentru rularea serverului SpringBoot
- "Node.js 22.19" și "npm 11.12": pentru rularea interfeței de utilizator
- "PostgreSQL 15": aplicația nativă sau în container Docker
- "RabbitMQ 3": aplicația nativă sau în container Docker

După ce baza de date și coada de mesaje sunt pornite, se configurează conexiunile către acestea în "application.properties" (Vezi 5.2.2) și se pornește serverul SpringBoot cu comanda "mvn spring-boot:run" în folderul "backend".

După ce serverul SpringBoot rulează, se configurează conexiunea către acesta în fișierul "vite.config.js" (Vezi 5.3.2) și se instalează dependențele interfeței de utilizator cu comanda "npm install" în folderul "frontend". După instalare, se rulează serverul local Vite cu comanda "npm run dev", apoi se accesează aplicația la adresa "http://localhost:3000".

Pentru a conecta telefonul la aplicația care rulează pe calculatorul local, trebuie mai întâi ca acestea să fie în aceeași rețea permisivă. Pentru a putea scana o adresă validă din codul QR și pentru a putea accesa funcționalitățile aplicației, trebuie schimbate următoarele variabile din fișierul "docker-compose.yml":

- "APP_BASE_URL" : localhost se înlocuiește cu adresa IP a calculatorului
- "APP_CORS_ALLOWED_ORIGINS": se adaugă în listă adresa IP a calculatorului cu portul 3000

După aceste modificări, aplicația se accesează de pe telefon la adresa "http://<IP_CALCULATOR>:3000".

7.2 Utilizarea aplicației

Primul contact cu aplicația este pagina de autentificare. Un utilizator nou își creează un cont din pagina de înregistrare, după care se poate autentifica. Cele două pagini se observă în figura 7.1.

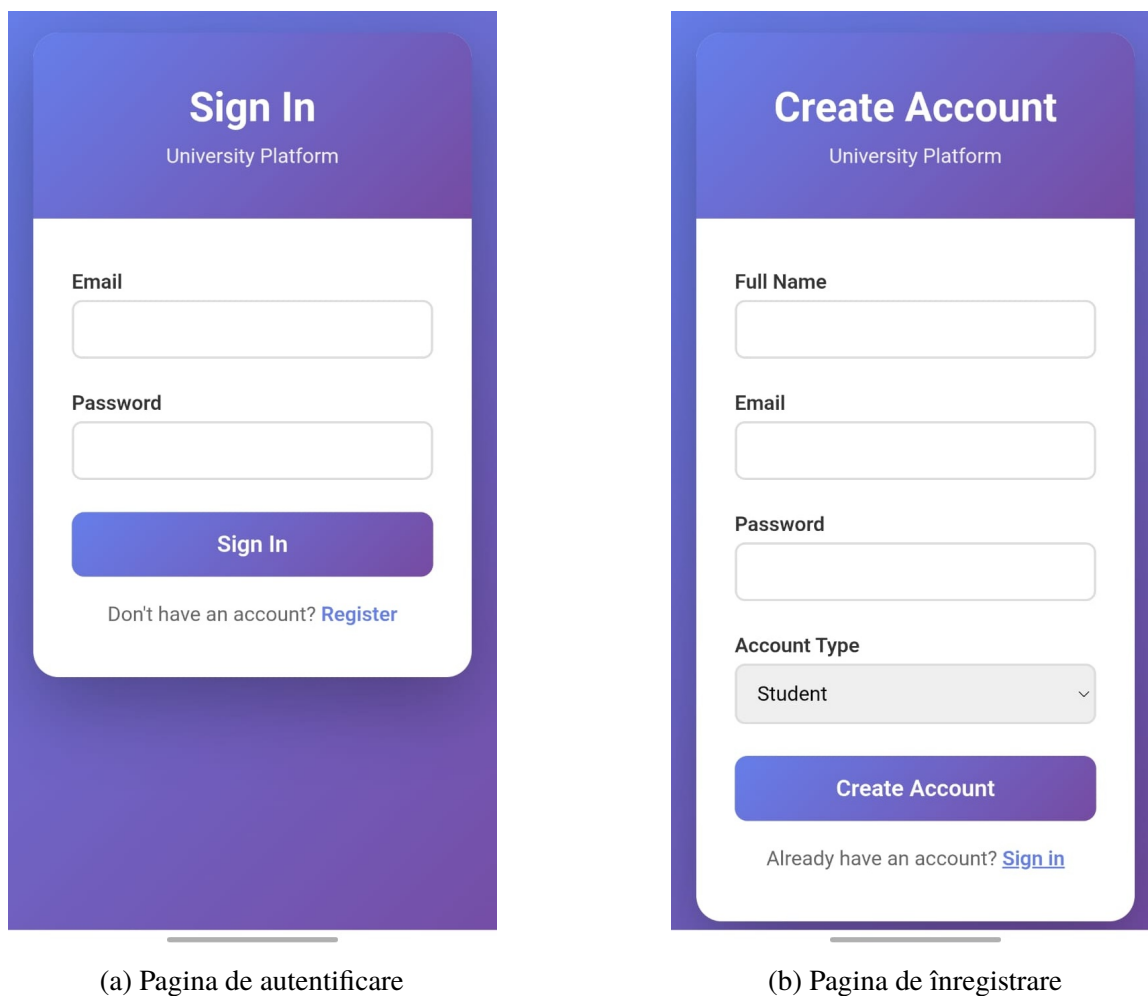


Figura 7.1: Paginile de autorizare

După autentificare, profesorul ajunge la pagina de pornire a sesiunilor de curs. De aici el alege un curs și pornește o sesiune apăsând butonul "Start Session", după cum se vede în figura 7.2.

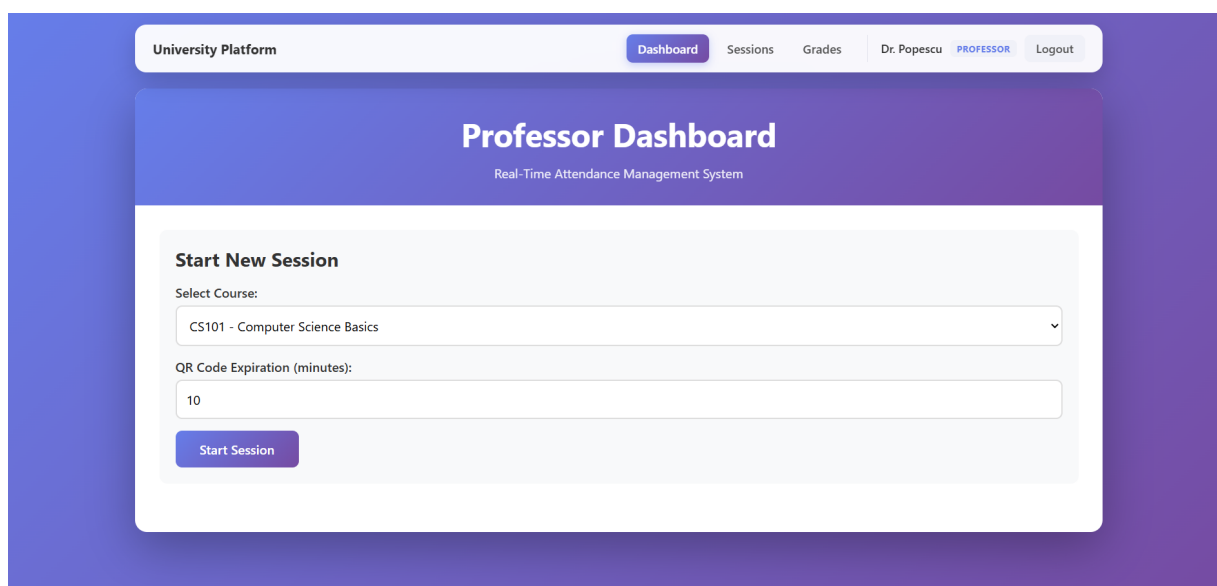


Figura 7.2: Pornirea sesiunii de curs

Odată pornită sesiunea, se derulează pagina în jos unde apare codul QR pe care studenții îl scanează. Sub cod se afișează lista cu studenții prezenți care se actualizează în timp real pe măsură ce aceștia se înregistrează. Acest lucru se observă în figura 7.3.

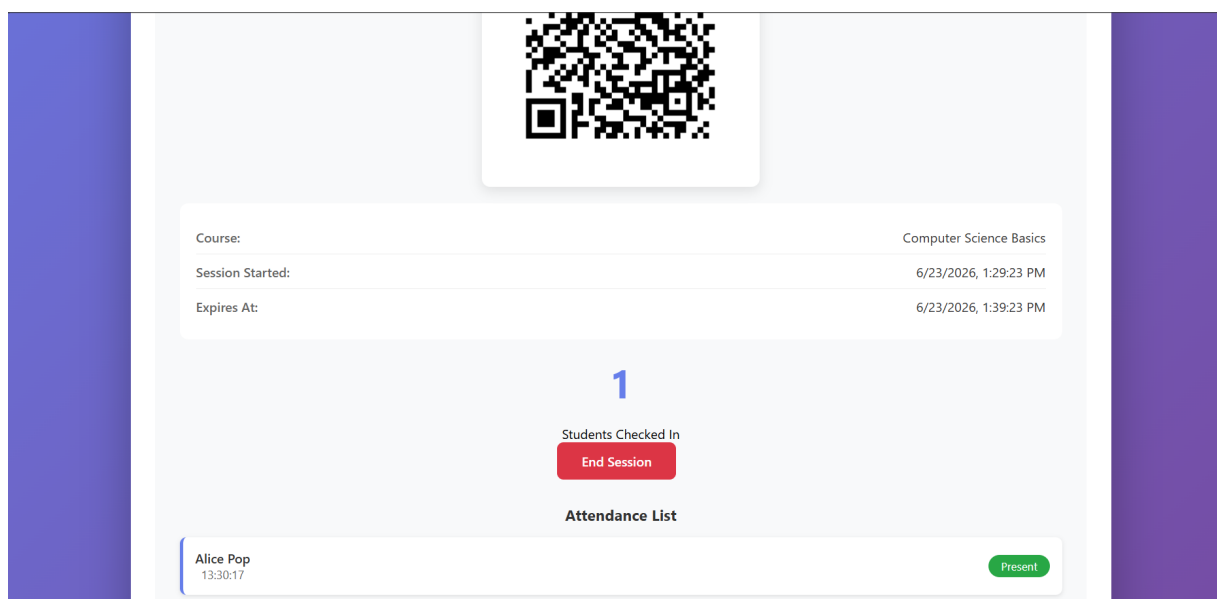


Figura 7.3: Codul QR și prezența în timp real

Pentru a ajunge la alte pagini dedicate profesorului, se utilizează bara de navigare din partea de sus a paginii. Pagina de istoric a sesiunilor este prezentată în figura 7.4.

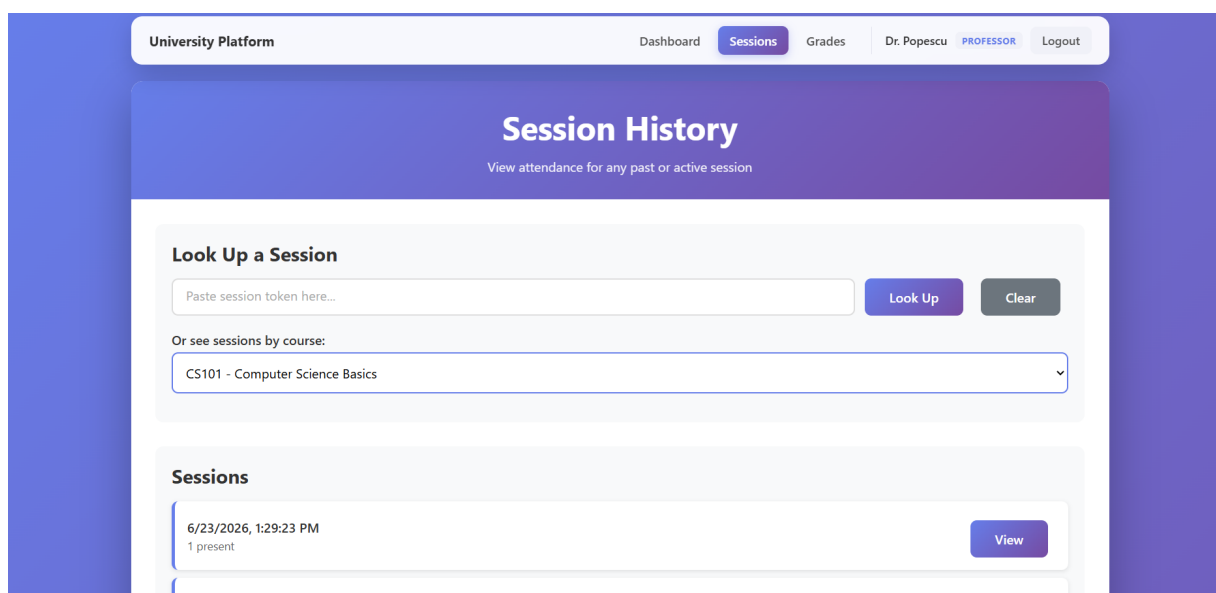


Figura 7.4: Istoricul sesiunilor de curs

Pe lângă prezență, profesorul gestionează și notele studenților. În pagina prezentată în figura 7.5 se pot adăuga sau modifica notele pentru fiecare student și pentru fiecare curs. De asemenea, se poate observa rata de prezență individuală a studenților pentru cursul ales și se poate face căutarea studenților după nume, email sau identificator.

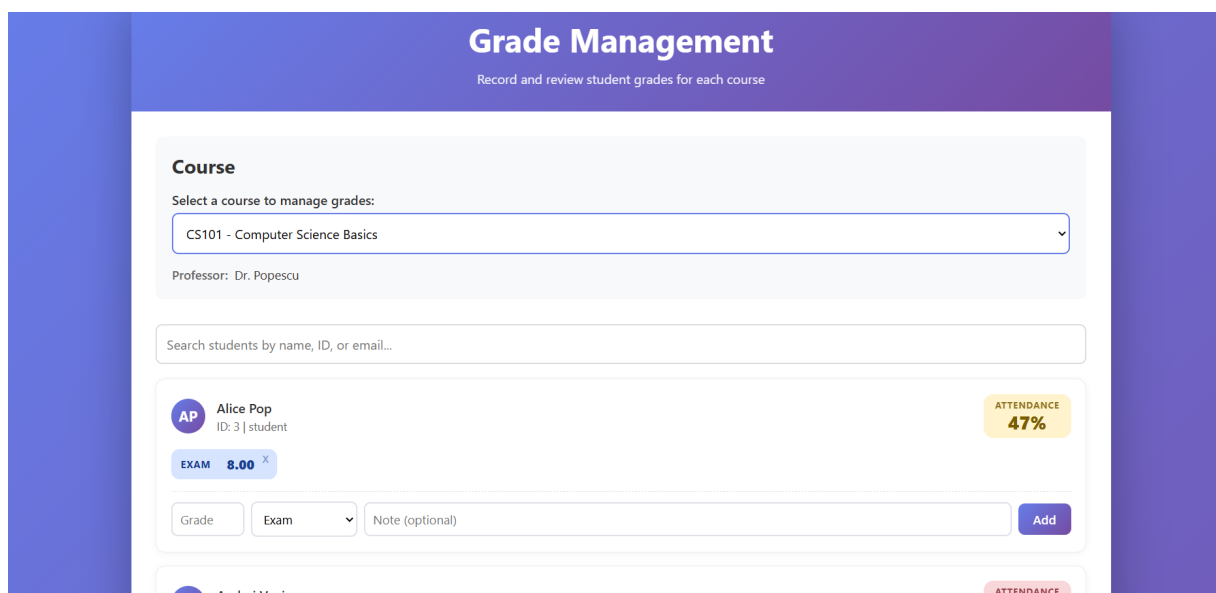


Figura 7.5: Gestionarea notelor

Scanarea codului QR generat anterior direcționează studentul la o pagină de confirmare a prezenței. După confirmare, prezența este înregistrată și studentul este redirecționat către paginile personale de vizualizare a notelor și a prezențelor. Aceste trei pagini se observă în figura 7.6.

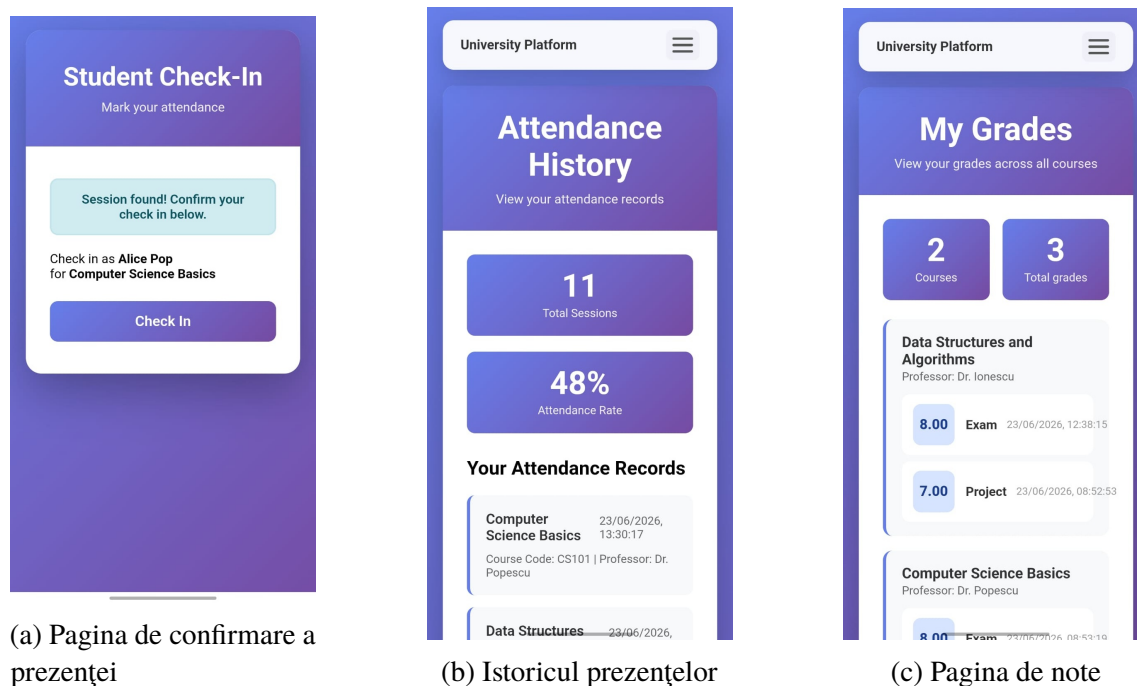


Figura 7.6: Paginile pentru studenți

Pagina administratorului este prezentată în figura 7.7. Se alege tabelul dorit și se execută comenzile către baza de date.

University Platform					
Admin User ADMIN Logout					
Users Courses Sessions Attendance Grades					
ID	NAME	EMAIL	ROLE	CREATED	ACTIONS
4	Ion Popescu	ion@university	STUDENT	6/19/2026, 9:47:05 AM	Edit Delete
5	Andrei Vanic	andrei@university	STUDENT	6/19/2026, 9:47:05 AM	Edit Delete
7	Evea Stann	eva@university	STUDENT	6/19/2026, 9:47:05 AM	Edit Delete
8	Ovidiu	ovidiu@o	STUDENT	6/19/2026, 10:08:52 AM	Edit Delete
6	David Brumaru	david@university	STUDENT	6/19/2026, 9:47:05 AM	Edit Delete
3	Alice Pop	student	STUDENT	6/19/2026, 9:47:05 AM	Edit Delete
2	Dr. Popescu	professor	PROFESSOR	6/19/2026, 9:47:05 AM	Edit Delete
1	Admin User	admin	ADMIN	6/19/2026, 9:47:05 AM	Edit Delete

Figura 7.7: Pagina de administrare

Capitolul 8. Concluzii

Lucrarea de față a pornit de la o problemă concretă din mediul educațional: înregistrarea prezenței la cursuri se face de obicei manual, consumă timp și nu oferă date centralizate. Pentru a rezolva această problemă s-a proiectat și s-a implementat o platformă web completă, în care profesorii pot să gestioneze prezența cursurilor prin coduri QR și au de asemenea și unelte de gestionare a notelor integrate.

Pe parcursul lucrării s-au parcurs toate etapele dezvoltării unui proiect software: s-a făcut o cercetare a soluțiilor și tehnologiilor existente, s-au stabilit obiectivele și specificațiile, s-a proiectat arhitectura iar apoi s-a trecut la implementarea propriu-zisă. Contribuțiile personale din acest proiect sunt următoarele:

- Proiectarea arhitecturii pe trei niveluri a aplicației, care separă interfața de utilizator, funcționalitățile serverului și infrastructura
- Implementarea serverului SpringBoot pe straturi (controlere, servicii, depozite), cu endpoint-uri REST separate pe roluri
- Realizarea unui mecanism de autentificare bazat pe token de tip UUID și interceptarea cererilor
- Implementarea procesului de înregistrare a prezenței prin procesare asincronă folosind coada de mesaje RabbitMQ
- Implementarea actualizărilor în timp real prin WebSocket
- Construirea interfeței de utilizator cu pagini separate pe roluri
- Proiectarea bazei de date și containerizarea întregii aplicații cu Docker
- Scrierea testelor unitare, de integrare și de performanță
- Asigurarea funcționalității aplicației

8.1 Rezultate obținute

Rezultatul obținut este o platformă universitară web completă care rezolvă problema inițială. Obiectivele specifice propuse la începutul lucrării au fost atinse astfel:

- Gestionarea sesiunilor de curs: profesorul poate porni și opri o sesiune, iar sistemul generează automat codul QR corespunzător sesiunii
- Înregistrarea prezenței prin cod QR: studentul scanează codul și este dus la pagina de înregistrare de unde confirmă și trimite cererea de înregistrare a prezenței
- Vizualizarea în timp real: lista studenților prezenți se actualizează prin WebSocket pe ecranul profesorului
- Gestionarea notelor: profesorii introduc sau modifică note iar studenții le văd în conturile lor proprii
- Istoricul sesiunilor: atât profesorii cât și studenții au acces la sesiunile anterioare
- Pagina de administrare: administratorul poate gestiona direct datele din baza de date
- Implementarea unui sistem de autentificare: utilizatorul se conectează, iar sistemul stabilește rolul și nivelul de acces

Pe lângă obiectivele funcționale specifice, specificațiile tehnice au fost și ele îndeplinite. Cele mai importante dintre acestea au fost verificate prin testare automată. Testarea unitară a confirmat logica componentelor importante iar testarea de integrare a verificat comunicarea între aceste componente. Rezultatele testului de performanță rulat în mai multe contexte este validarea care justifică pe deplin arhitectura cu coada de mesaje aleasă.

De asemenea, au fost îndeplinite și specificațiile non-funcționale. Arhitectura proiectului permite scalabilitate, portabilitate și mentenabilitate prin tehnologiile Docker și RabbitMQ utilizate. Accesul la resurse este controlat prin autentificare iar aplicația este ușor de folosit, intuitivă din perspectiva utilizatorului și robustă.

8.2 Direcții de dezvoltare

Aplicația rezolvă problema propusă, însă există multe direcții de dezvoltare care ar putea aduce îmbunătățiri. În continuare se prezintă aceste direcții, împărțite în două categorii: direcțiile conceptuale de dezvoltare și direcțiile de dezvoltare pentru o implementare concretă într-un mediu universitar real.

Direcțiile de dezvoltare conceptuale aduc un plus de valoare aplicației, indiferent de modul concret de adopție a acesteia. Acestea sunt următoarele:

- Înlocuirea mecanismului de autentificare actual cu unul bazat pe JWT și pe cadrul de lucru Spring Security
- Adăugarea unor măsuri anti-fraudă la înregistrarea prezenței, astfel încât un student să nu poată înregistra prezența fără să fie fizic în sala de curs
- Integrarea unui calendar sau a unui orar pentru sălile de curs și pentru examene

- Trimiterea de notificări către studenți
- Adăugarea unei pagini cu detaliile personale ale utilizatorului

Pe adoptarea aplicației într-un mediu universitar real există direcții de dezvoltare care au nevoie de decizii sau informații concrete din acest mediu. Direcțiile de dezvoltare din această categorie sunt următoarele:

- Automatizarea creării conturilor de student: conturile s-ar putea genera automat, cu o parolă implicită trimisă pe email. O variantă alternativă ar fi ca fiecare cont nou să treacă printr-un pas de verificare
- Gruparea studenților pe ani și pe grupe, împreună cu asocierea cursurilor anumitor ani universitari
- Asocierea profesorilor cu propriile cursuri, astfel încât fiecare profesor să aibă control doar asupra cursurilor pe care le are în custodie
- Implementarea unui deployment cu setările de configurare adaptate pentru expunerea aplicației pe internet

Bibliografie

- [1] F. Masalha and N. Hirzallah, “A students attendance system using qr code,” *International Journal of Advanced Computer Science and Applications*, 2014.
- [2] M. Ng, “Concept and design of a smart classroom attendance system,” in *Proceedings of the International Conference on Education and Information Technology*, 2014.
- [3] Denso Wave Incorporated, “QR Code essentials,” <https://www.qrcode.com/en/>, 2011.
- [4] International Organization for Standardization, “ISO/IEC 18004:2015 – Information technology – Automatic identification and data capture techniques – QR Code bar code symbology specification,” Standard, ISO, 2015.
- [5] VMware, “Spring Boot Reference Documentation,” <https://docs.spring.io/spring-boot/index.html>, 2024.
- [6] Django Software Foundation, “Django Documentation,” <https://docs.djangoproject.com/>, 2024.
- [7] S. Ramírez, “FastAPI Documentation,” <https://fastapi.tiangolo.com/>, 2024.
- [8] Mozilla Developer Network, “HTML: HyperText Markup Language,” <https://developer.mozilla.org/en-US/docs/Web/HTML>, 2024.
- [9] —, “CSS: Cascading Style Sheets,” <https://developer.mozilla.org/en-US/docs/Web/CSS>, 2024.
- [10] Meta Open Source, “React Documentation,” <https://react.dev/>, 2024.
- [11] Google, “Angular Documentation,” <https://angular.dev/>, 2024.
- [12] Mozilla Developer Network, “An overview of HTTP,” <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>, 2024.
- [13] R. T. Fielding and J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,” RFC 7231, Internet Engineering Task Force, 2014.
- [14] Mozilla Developer Network, “API (Application Programming Interface),” <https://developer.mozilla.org/en-US/docs/Glossary/API>, 2024.

- [15] H. Mann, “Rest api basics - 4 things you need to know,” <https://mannhowie.com/rest-api/>, 2023.
- [16] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [17] T. Berners-Lee, R. T. Fielding, and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax,” RFC 3986, Internet Engineering Task Force, 2005.
- [18] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force, 2017.
- [19] Mozilla Developer Network, “SPA (Single-page application),” <https://developer.mozilla.org/en-US/docs/Glossary/SPA>, 2024.
- [20] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, 2011.
- [21] STOMP, “STOMP Protocol Specification, Version 1.2,” <https://stomp.github.io/stomp-specification-1.2.html>, 2012.
- [22] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [23] F. Oladipupo, “Optimizing api performance part 3: Asynchronous processing & queues,” https://dev.to/fredrick_oladipupo/optimizing-api-performance-part-3-asynchronous-processing-queues-h1d, 2025.
- [24] OASIS, “Advanced Message Queuing Protocol (AMQP) Version 1.0,” OASIS Standard, 2012.
- [25] Docker, Inc., “Docker Documentation,” <https://docs.docker.com/>, 2024.
- [26] ———, “Docker Compose Documentation,” <https://docs.docker.com/compose/>, 2024.
- [27] The Kubernetes Authors, “Kubernetes Documentation,” <https://kubernetes.io/docs/home/>, 2024.
- [28] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, 1970.
- [29] The PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” <https://www.postgresql.org/docs/>, 2023.
- [30] Oracle Corporation, “MySQL 8.0 Reference Manual,” <https://dev.mysql.com/doc/refman/8.0/en/>, 2024.

- [31] A. P. Singh, “Client-server architecture explained,” <https://blog.algomaster.io/p/client-server-architecture-explained>, 2025.
- [32] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.
- [33] “What are microservices?” <https://www.akamai.com/glossary/what-are-microservices>.
- [34] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley, 2014.
- [35] T. Lindholm, F. Yellin, G. Bracha, and A. Buckley, *The Java Virtual Machine Specification, Java SE 8 Edition*. Addison-Wesley, 2014.
- [36] The Apache Software Foundation, “Apache Maven Documentation,” <https://maven.apache.org/>, 2024.
- [37] The Project Lombok Authors, “Project Lombok,” <https://projectlombok.org/>, 2024.
- [38] VMware, “Spring Framework Reference Documentation,” <https://docs.spring.io/spring-framework/reference/>, 2024.
- [39] R. Johnson, *Expert One-on-One J2EE Design and Development*. Wrox Press, 2002.
- [40] VMware, “Spring Framework – WebSocket Support,” <https://docs.spring.io/spring-framework/reference/web/websocket.html>, 2024.
- [41] ———, “Spring Data JPA Reference Documentation,” <https://docs.spring.io/spring-data/jpa/reference/>, 2024.
- [42] ZXing Authors, “ZXing (“Zebra Crossing”) barcode scanning library for Java, Android,” <https://github.com/zxing/zxing>, 2024.
- [43] Broadcom, “RabbitMQ – AMQP 0-9-1 Model Explained,” <https://www.rabbitmq.com/tutorials/amqp-concepts>, 2024.
- [44] F5, Inc., “nginx documentation,” <https://nginx.org/en/docs/>, 2024.